

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a0baf7ebe53c21630.jsonl`
- SHA-256: `83ba1b7f318a2b87f1c71e25a59bec7805edd711347b250ee5a217563429b4bf`
- Source modified: `2026-07-08T06:11:46+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:04:07.731Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

Your review dimension: regression.

Review the #521 diff for REGRESSIONS to the default-OFF path and existing behaviour. Confirm: an unset phase_placement + compute_target='' still stitches in-process and validates on the control-plane byte-identically; _maybe_dispatch_remote's default target branch still dispatches exactly when compute_target==cloud_run; the _execute_processing validation-gating

(validate_on_control_plane) can NEVER skip validation when nobody would validate;
VideoCompiler._trim_one_segment still behaves identically for the default libx264 encoder.

2. assistant (2026-07-08T06:04:13.223Z)

I'll start by understanding the environment and finding the actual diff/source, since the paths came through as "undefined".

3. user (2026-07-08T06:04:15.649Z)

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (\$9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag (#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed + downsample fix proven (#504)
bd5647b fix(worker): surface detector failure/timeout as failed, not silent 0-rally success (#503)

4. user (2026-07-08T06:04:17.081Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521)
63d65f9 fix(cloud-serving): serve large reels via a gs:// signed URL (#463) (#469)
a5d6246 serving: re-queue interrupted compile jobs on restart (#329 reliability) (#460)
b09a1d4 serving: re-queue interrupted compile jobs on restart (#329 reliability)
739d1f4 fix: Q2 motion return type + D11 compiler codec docstring (#362)
2c1028a docs(guide): add CREATING_REELS.md ? end-user install?reel walkthrough (#343)
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329) (#339)
fa2342c feat(nightly): E2E engine reel-count regression on a GCP GPU VM (emailed) + cost (#315)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
be0c434 feat(pkg): P1a ? net-new console/wizard endpoints: /api/version, /api/capabilities, /api/videos, /api/reels (#300)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching (#289)
e6110ce feat(api): GET /api/health + return download_url from /api/compile (M1/P6) (#268)
d8c5aa0 docs(deploy): report polish ? highlight reel smoke-test result + VM-deleted final state (#262)
3f5a56c fix(gemini): surface failed chunks loudly (no more silent partial reels) (#240)
a3b4954 feat(runner): --min-rally-duration flag (surface the long-rally reel filter on the CLI) (#238)
20f8aa3 feat(eval): P7 long-rally (>5s) quality lens + reel knob (both default-OFF) (#220)
18883b5 refactor(stitcher): extract per-segment trim into _trim_one_segment (#224)
8736e2d fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels (#205)
4942d51 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation (#182)

562308b fix(stitcher): merge overlapping/duplicate segments so a reel never stitches the same rally twice (#180)

428ee44 address follow-up review on #169: correct factual error in item-3 investigation note (fusion_compare.py, fusion_golden.py, ablation.py + tests *do* exist per ls/grep on the tree; person features in fusion_hybrid + detectors/fusion_features.py; remaining M0 in-container task is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan now accurate. M0 item 3 placement remains correct and even stronger.

318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys warning, get shim, roundtrip + living doc update for T2 post #157

8836dfe chore: scripts/ for run-drivers + CODE_MAP \$0 file-placement guidelines (#138)

6de70aa fix(stitcher): escape drive-letter colon in ffmpeg filtergraph paths (#108)

3461898 feat(api): enforce stitching.min_shot_count on /api/compile (#107)

112e75d feat(stitcher): bundle Apache-2.0 default watermark font + log fallback reason

82e479f Merge pull request #102 from avidullu:fix/stitching-padding-config

30e4899 fix(api): thread config stitching paddings into live VideoCompiler sites

10864f3 feat(stitcher): configurable branding watermark on highlight reels (#101)

5. assistant (2026-07-08T06:04:22.363Z)

The task's paths came through as "undefined", but the change is merged. The PR #521 commit is `8866ffe` (merged as `dac6736`). Let me get the actual diff and the current source.

6. user (2026-07-08T06:04:25.176Z)

commit 8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f
 Author: Avi Dullu <avi.dullu@gmail.com>
 Date: Wed Jul 8 11:19:30 2026 +0530

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, \$9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download_url} shape the SPA expects. The worker stitches via the SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
 Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-Author-By: Claude Opus 4.8 <noreply@anthropic.com>

backend/compute/__init__.py	2 +
backend/compute/base.py	34 +++
backend/compute/cloud_run.py	100 ++++++-
backend/config/__init__.py	16 ++
backend/config/models.py	52 ++++
backend/config/placement.py	59 ++++
backend/config/presets.py	10 +-
backend/orchestration.py	322 ++++++++-----
backend/pipeline/stitcher/compiler.py	94 ++++++-
backend/worker/__main__.py	139 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md	169 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md	1 +
tests/test_api_cloud_dispatch.py	70 +++++
tests/test_compile_dispatch.py	204 ++++++++
tests/test_compiler.py	64 +++++
tests/test_compute_backend.py	111 ++++++
tests/test_config_deployment.py	5 +
tests/test_phase_placement.py	138 ++++++++
tests/test_worker.py	175 ++++++++

7. user (2026-07-08T06:04:27.876Z)

error: Your local changes to the following files would be overwritten by merge:

```

backend/config/models.py
backend/orchestration.py
backend/pipeline/stitcher/compiler.py
tests/test_compiler.py

```

8. user (2026-07-08T06:04:34.565Z)

On branch codex/issue-523-sd-reel-plan

Your branch is behind 'origin/master' by 1 commit, and can be fast-forwarded.

(use "git pull" to update your local branch)

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

```

modified:   README.md
modified:   SESSION_HANDOFF.md
modified:   backend/config/models.py
modified:   backend/infrastructure/database_jobs.py
modified:   backend/main.py
modified:   backend/orchestration.py
modified:   backend/pipeline/stitcher/compiler.py
modified:   config.json
modified:   tests/test_api_endpoints.py
modified:   tests/test_async_jobs.py
modified:   tests/test_compiler.py
modified:   tests/test_watermark.py

```

Untracked files:

(use "git add <file>..." to include in what will be committed)

```

score_golden_gate.py
score_local.py

```

no changes added to commit (use "git add" and/or "git commit -a")

=== HEAD ===

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (§9) (#522)

=== remote ===

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

9. assistant (2026-07-08T06:04:49.733Z)

I'm on a sibling session's WIP branch (`codex/issue-523...`) with uncommitted changes to the very files #521 touched. Per safety rules I won't touch the working tree. I'll read the exact post-#521 source from the merged commit instead. Let me get the full diff.

10. user (2026-07-08T06:04:55.215Z)

2168 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/pr521.diff
parent: 699d85bc0abd6e35d3196a4566dcec23f649394e

```
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
```

11. user (2026-07-08T06:04:59.362Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1061 of 2169 total (43403 tokens, cap 25000). Call Read with offset=1062 limit=1061 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1      parent: 699d85bc0abd6e35d3196a4566dcec23f649394e
2
3      diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
4      index 482f3c9..7f8663f 100644
5      --- a/backend/compute/__init__.py
6      +++ b/backend/compute/__init__.py
7      @@ -14,6 +14,7 @@ import os
8       from typing import Any, Optional
9
10     from backend.compute.base import (
11 +     CompileJobSpec,
12         ComputeBackend,
13         ComputeJobSpec,
14         ComputeResult,
15     @@ -24,6 +25,7 @@ from backend.infrastructure.execution_policy import DEFAULT_POLICY,
16     ExecutionPol
17     __all__ = [
18         "ComputeBackend",
19     +     "CompileJobSpec",
20         "ComputeJobSpec",
21         "ComputeResult",
22         "RemoteExecutionFailed",
23         "get_compute_backend",
24     diff --git a/backend/compute/base.py b/backend/compute/base.py
25     index 3ad76fc..f7afb40 100644
26     --- a/backend/compute/base.py
27     +++ b/backend/compute/base.py
28     @@ -51,6 +51,24 @@ class ComputeJobSpec:
29         skip_validation: bool = False
30
31     +@dataclass(frozen=True)
32     +class CompileJobSpec:
33     +    """One reel-COMPILE job dispatched to the worker (#521). The heavy input ? the
34     resolved
35     34     +    ``(start, end)`` time-pairs plus the control-plane's stitching/watermark config and
36     the source
37     35     +    URI ? is staged to ``spec_uri`` (a small JSON in the bucket) rather than crammed
38     onto argv, so
39     36     +    the worker needs no control-plane DB: it reads the spec, fetches the source,
40     stitches on the L4
41     37     +    (NVENC), uploads the reel to ``<out_uri>/highlights/<video_id>/<output_name>``, and
42     writes a
43     38     +    done-marker for the dispatcher's bucket-poll."""
44     39     +
45     40     +    out_uri: str # output ROOT (gs://? bucket); highlights/<video_id>/<name> is
```

```

written under it
41 + spec_uri: str # staged compile-spec JSON (time-pairs + stitching + source uri +
video_id)
42 + video_id: str # the reel's video_id (deterministic reel path + marker)
43 + output_name: str # traversal-safe reel filename
44 + # Raw worker ``--set k=v`` overrides (storage.output_root/backend +
stitching.encoder). A tuple
45 + # (not list/dict) so the spec stays frozen/hashable.
46 + config_overrides: tuple[str, ...] = ()
47 +
48 +
49 @dataclass(frozen=True)
50 class ComputeResult:
51     """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
52 @@ -85,3 +103,19 @@ class ComputeBackend(ABC):
53     ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend
is
54     still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
55     long GPU run doesn't look frozen. Implementations without a wait loop may
ignore it."""
56 +
57 + def run_compile(
58 +     self,
59 +     spec: "CompileJobSpec",
60 +     *,
61 +     on_wait: "Callable[[float], None] | None" = None,
62 + ) -> ComputeResult:
63 +     """Run a reel-COMPILE ``spec`` to completion and return its ``ComputeResult``
(#521).
64 +
65 +     Non-abstract with a NotImplementedError default: only backends that can offload
the ffmpeg
66 +     stitch (``CloudRunCompute``) override it ? an in-process/other backend never
dispatches a
67 +     compile. The returned ``ComputeResult.returncode`` is the worker's rc (0 ok /
non-zero =
68 +     stitch failure) and ``report`` is the worker's done-marker (carries the reel's
download_url)."""
69 +     raise NotImplementedError(
70 +         f"{type(self).__name__} does not support remote compile dispatch"
71 +     )
72 diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
73 index a4a0743..8df4246 100644
74 --- a/backend/compute/cloud_run.py
75 +++ b/backend/compute/cloud_run.py
76 @@ -37,6 +37,7 @@ from abc import ABC, abstractmethod
77     from typing import Any, Callable, List, Optional
78
79     from backend.compute.base import (
80 +         CompileJobSpec,
81 +         ComputeBackend,
82 +         ComputeJobSpec,
83 +         ComputeResult,
84 @@ -278,21 +279,7 @@ class CloudRunCompute(ComputeBackend):
85         done_uri,
86     )
87
88     - try:
89     -     marker = poll_until(
90     -         lambda: self._poll_check(done_uri, str(execution or "")),
91     -         self._policy,
92     -         label="cloud-run-infer",
93     -         logger=LOG,

```

```

94         -             # Live heartbeat (#482): each still-waiting tick reaches the caller so
95         -             # job.progress moves during the whole GPU run instead of freezing.
96         -             on_tick=on_wait,
97         -         )
98         -     except PolicyTimeout as timeout_exc:
99         -         # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
100         -             # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
101         -             # clearly-labelled failure instead of the ~65-min opaque timeout this
branch handles.
102         -             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
103         +             marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
infer")
104             video_id = str(marker.get("video_id", ""))
105             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
106             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
107     @@ -317,6 +304,89 @@ class CloudRunCompute(ComputeBackend):
108         execution=str(execution or ""),
109     )
110
111     +     def run_compile(
112     +         self,
113     +         spec: "CompileJobSpec",
114     +         *,
115     +         on_wait: "Callable[[float], None] | None" = None,
116     +     ) -> ComputeResult:
117     +         """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
118     +         bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
119     +         dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast
+ timeout
120     +         rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
121     +         ~65-min opaque hang."""
122     +         out_root = spec.out_uri.rstrip("/")
123     +         token = self._token or uuid.uuid4().hex
124     +         # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
125     +         # bucket can never collide on a marker.
126     +         done_uri = f"{out_root}/_compile/{token}.done"
127     +         argv: List[str] = [
128     +             "compile",
129     +             "--out-uri",
130     +             spec.out_uri,
131     +             "--spec-uri",
132     +             spec.spec_uri,
133     +             "--done-uri",
134     +             done_uri,
135     +         ]
136     +         # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
137     +         # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
138     +         for kv in spec.config_overrides:
139     +             argv += ["--set", kv]
140     +
141     +         execution = self._client.run_job(
142     +             self._job_name, argv, region=self._region, project=self._project
143     +         )
144     +         LOG.info(
145     +             "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",

```

```

146 +         self._job_name,
147 +         execution,
148 +         done_uri,
149 +     )
150 +     marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
151 +     video_id = str(marker.get("video_id", "") or spec.video_id)
152 +     rc = int(marker.get("returncode", 1))
153 +     LOG.info(
154 +         "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
155 +         self._job_name,
156 +         video_id,
157 +         rc,
158 +     )
159 +     # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
160 +     # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
161 +     # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
162 +     return ComputeResult(
163 +         returncode=rc,
164 +         outputs_uri=str(marker.get("reel_uri", "") or ""),
165 +         video_id=video_id,
166 +         report=marker,
167 +         execution=str(execution or ""),
168 +     )
169 +
170 +     def _await_marker(
171 +         self,
172 +         done_uri: str,
173 +         execution: object,
174 +         on_wait: "Callable[[float], None] | None",
175 +         *,
176 +         label: str,
177 +     ) -> dict:
178 +         """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer``
and
179 +         ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting
tick reaches the
180 +         caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
181 +         ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
182 +         maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
183 +         try:
184 +             return poll_until(
185 +                 lambda: self._poll_check(done_uri, str(execution or "")),
186 +                 self._policy,
187 +                 label=label,
188 +                 logger=LOG,
189 +                 on_tick=on_wait,
190 +             )
191 +         except PolicyTimeout as timeout_exc:
192 +             return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
193 +
194 +     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
195 +         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
196
197 diff --git a/backend/config/__init__.py b/backend/config/__init__.py
198 index 3f722c9..14cf483 100644
199 --- a/backend/config/__init__.py
200 +++ b/backend/config/__init__.py
201 @@ -19,10 +19,19 @@ from .models import ( # re-export common ones

```

```

202         Config,
203         DeploymentConfig,
204         IndexingConfig,
205     +   PhasePlacementConfig,
206         StitchingConfig,
207         StorageConfig,
208         ValidationConfig,
209     )
210 +from .placement import (
211     +   PHASE_COMPILE,
212     +   PHASE_SEGMENT,
213     +   PHASE_VALIDATION,
214     +   PLACE_CLOUD_L4,
215     +   PLACE_CONTROL_PLANE,
216     +   resolve_phase_placement,
217 +)
218     from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
219     from .startup import (
220         StartupCheck,
221     @@ -52,9 +61,16 @@ __all__ = [
222         "Config",
223         "DeploymentConfig",
224         "IndexingConfig",
225     +   "PhasePlacementConfig",
226         "StitchingConfig",
227         "StorageConfig",
228         "ValidationConfig",
229     +   "resolve_phase_placement",
230     +   "PHASE_VALIDATION",
231     +   "PHASE_SEGMENT",
232     +   "PHASE_COMPILE",
233     +   "PLACE_CONTROL_PLANE",
234     +   "PLACE_CLOUD_L4",
235         "PROFILE_PRESETS",
236         "suggest_profile",
237         "probe_cuda_available",
238     diff --git a/backend/config/models.py b/backend/config/models.py
239     index a7c997c..ce8e98a 100644
240     --- a/backend/config/models.py
241     +++ b/backend/config/models.py
242     @@ -364,6 +364,14 @@ class StitchingConfig(BaseModel):
243         # background-court fragments), so this keeps the reel to the eye-catching long
244         rallies.
245         # OFF by default ? the detector output is unchanged, only which rallies reach the
246         reel.
247         min_rally_duration: float = Field(default=0.0, ge=0.0)
248     +   # Video encoder for the per-clip trim re-encode (the branding watermark forces a
249     +   re-encode; the
250     +   # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
251     +   ``h264_nvenc`` /
252     +   # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-
253     +   encode that
254     +   # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested
255     +   encoder ONCE and
256     +   # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never
257     +   breaks. The
258     +   # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for
259     +   the L4 run;
260     +   # the control-plane's own local stitch keeps libx264.
261     +   encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
262     +   watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
263
264     @@ -451,6 +459,21 @@ class StorageConfig(BaseModel):
265         gdrive_api: dict[str, Any] = Field(default_factory=dict)

```

```

259
260
261     +class PhasePlacementConfig(BaseModel):
262     +     """Per-phase machine placement (#521): move validation / segment(detect) /
compile(stitch)
263     +     between the control-plane and the L4 Cloud Run worker **by config, no code
change**.
264     +
265     +     Each field is ``None`` (? the legacy default for that phase ? see
266     +     ``backend/config/placement.py``) or an explicit ``"control_plane"`` /
``"cloud_run_l4"``.
267     +     ``control_plane`` runs the phase in-process on the CPU-only control-plane;
``cloud_run_l4``
268     +     dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521
behaviour.
269     +     Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read
these directly."""
270     +
271     +     validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
272     +     segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
273     +     compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
274     +
275     +
276     class DeploymentConfig(BaseModel):
277     +     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
278     +     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
279     +     @@ -481,6 +504,35 @@ class DeploymentConfig(BaseModel):
280     +     +     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
281     +     +     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
282     +     +     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
283     +     +     # Per-phase machine placement (#521). Unset ? today's placement (validation +
compile on the
284     +     +     # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to
relocate it by
285     +     +     # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read
the sub-fields
286     +     +     # directly ? the resolver folds in the legacy compute_target default).
287     +     +     phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
288     +     +     # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for
the L4 stitch
289     +     +     # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
290     +     +     # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is
placed on the L4;
291     +     +     # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264
by default).
292     +     +     cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
293     +     +
294     +     @model_validator(mode="after")
295     +     def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
296     +     +     # A clip can only be validated ON the worker if the worker also DETECTS it
there ? the worker
297     +     +     # is only launched for the segment(detect) phase. So validation=cloud_run_l4
with segment on
298     +     +     # the control-plane would leave NOBODY validating (the control-plane skips, and
no worker runs)
299     +     +     # ? reject that footgun at config load rather than ship a silently-unvalidated
clip.
300     +     +     if self.phase_placement.validation == "cloud_run_l4":
301     +     +         seg = self.phase_placement.segment or (
302     +     +             "cloud_run_l4" if self.compute_target == "cloud_run" else

```

```

"control_plane"
303     +         )
304     +         if seg != "cloud_run_l4":
305     +             raise ValueError(
306     +                 "deployment.phase_placement.validation='cloud_run_l4' requires
segment to also "
307     +                 "run on the L4 ? the worker can only validate a clip it also
detects. Set "
308     +                 "phase_placement.segment='cloud_run_l4' (or
compute_target='cloud_run'), or keep "
309     +                 "validation on the control_plane."
310     +             )
311     +         return self
312
313
314     class BillingConfig(BaseModel):
315     diff --git a/backend/config/placement.py b/backend/config/placement.py
316     new file mode 100644
317     index 00000000..cda3ca5
318     --- /dev/null
319     +++ b/backend/config/placement.py
320     @@ -0,0 +1,59 @@
321     +"""Phase ? machine placement resolution (#521).
322     +
323     +A *phase placement* answers one question per pipeline phase ? **where does the heavy
compute for
324     +this phase run?** ? with a config knob (`deployment.phase_placement`) instead of
code. The three
325     +placeable phases and their values:
326     +
327     +   validation ? control_plane | cloud_run_l4
328     +   segment    ? control_plane | cloud_run_l4   (a.k.a. "detect")
329     +   compile     ? control_plane | cloud_run_l4   (a.k.a. "stitch")
330     +
331     +`control_plane` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
`cloud_run_l4`
332     +dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already
uses). This module
333     +is the SINGLE source of truth so `backend.orchestration` (segment + compile) and the
worker never
334     +re-derive the mapping.
335     +
336     +**Default = today's behaviour (pure / additive).** When a phase's knob is unset
(`None`) the
337     +resolver reproduces the pre-#521 placement:
338     +
339     +   validation ? control_plane                                     (the authoritative
gate, #512/#513)
340     +   segment    ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
341     +   compile     ? control_plane                                     (the in-process
ffmpeg stitch)
342     +
343     +so a config with no `phase_placement` block behaves byte-identically to before this
module existed.
344     +"""
345     +
346     +from __future__ import annotations
347     +
348     +from typing import Any
349     +
350     +# Phase identities (kept as constants so callers don't stringly-type the phase names).
351     +PHASE_VALIDATION = "validation"
352     +PHASE_SEGMENT = "segment"
353     +PHASE_COMPILE = "compile"

```

```

354 +PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
355 +
356 +# Placement targets.
357 +PLACE_CONTROL_PLANE = "control_plane"
358 +PLACE_CLOUD_L4 = "cloud_run_l4"
359 +PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
360 +
361 +
362 +def resolve_phase_placement(cfg: Any, phase: str) -> str:
363 +     """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
364 +
365 +     Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the
legacy default
366 +     (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an
absent
367 +     ``deployment`` / ``phase_placement`` degrades to the legacy default.
368 +     """
369 +     deployment = getattr(cfg, "deployment", None)
370 +     placement = getattr(deployment, "phase_placement", None)
371 +     explicit = getattr(placement, phase, None) if placement is not None else None
372 +     if explicit:
373 +         return str(explicit)
374 +     # Legacy defaults ? reproduce today's placement exactly.
375 +     if phase == PHASE_SEGMENT:
376 +         compute_target = (getattr(deployment, "compute_target", "") or "").strip()
377 +         return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
378 +     # validation + compile default to the control-plane (their pre-#521 home).
379 +     return PLACE_CONTROL_PLANE
380 diff --git a/backend/config/presets.py b/backend/config/presets.py
381 index 77b20e8..0ea0c88 100644
382 --- a/backend/config/presets.py
383 +++ b/backend/config/presets.py
384 @@ -49,7 +49,15 @@ PROFILE_PRESETS: dict[str, dict[str, Any]] = {
385     "storage": {"backend": "local"},
386     },
387     "cloud-serving": {
388 -        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
389 +        "deployment": {
390 +            "profile": "cloud-serving",
391 +            "compute_target": "cloud_run",
392 +            # #521: run the compile/stitch on the L4 worker (NVENC + 8 vCPU), not the
CPU-only
393 +            # 2-vCPU control-plane where the 2026-07-08 CUJ measured ~13 min for an
11-clip 4K reel.
394 +            # (validation stays on the control-plane ? the authoritative #512/#513
gate; segment is
395 +            # already cloud_run_l4 via compute_target.)
396 +            "phase_placement": {"compile": "cloud_run_l4"},
397 +        },
398 +        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172
floor (D11).
399 +        # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the
paid L4
400 +        # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
diff --git a/backend/orchestration.py b/backend/orchestration.py
401 index 8b7851e..46ffaa4 100644
402 --- a/backend/orchestration.py
403 +++ b/backend/orchestration.py
404 @@ -26,11 +26,20 @@ import os
405 import re
406 import tempfile
407 import threading
408 +import uuid
409 from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
410 from urllib.parse import quote

```

```

412
413     from backend import storage
414     -from backend.config import AppConfig, StitchingConfig
415     +from backend.config import (
416     +     PHASE_COMPILE,
417     +     PHASE_SEGMENT,
418     +     PHASE_VALIDATION,
419     +     PLACE_CLOUD_L4,
420     +     AppConfig,
421     +     StitchingConfig,
422     +     resolve_phase_placement,
423     +)
424     from backend.pipeline.segmenters.base import segmenter_registry
425     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
426     from backend.pipeline.validators.base import ValidationResult, registry
427     @@ -206,6 +215,24 @@ def _cloud_available(cfg) -> bool:
428         return bool(has_job and out_root)
429
430
431     +def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
432     +     """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
433     +     ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
434     +     ``_execute_processing`` can
435     +     decide whether to DELEGATE validation to the worker
436     +     (``phase_placement.validation``). A per-request
437     +     picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
438     +     cloud iff
439     +     ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
440     +     pre-dispatch reject
441     +     (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
442     +     remotely."""
443     +     choice = (req_compute or "").strip().lower()
444     +     if choice == "local":
445     +         return False
446     +     if choice == "cloud":
447     +         return _cloud_available(cfg)
448     +     return (
449     +         resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
450     +         and _cloud_available(cfg)
451     +     )
452
453     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
454     +     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
455     +     minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
456     +     comes
457     +     @@ -505,11 +532,19 @@ def _maybe_dispatch_remote(
458     +         )
459     +         compute = get_compute_backend(cfg, target_override="cloud_run")
460     +     else:
461     +         compute = get_compute_backend(
462     +             cfg
463     +         ) # server default (default-OFF unless cloud_run)
464     +         # No explicit picker ? consult phase_placement.segment (#521). Its legacy
465     +         default is
466     +         # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
467     +         behaviour; setting
468     +         # phase_placement.segment relocates DETECT by config alone (e.g. force it back
469     +         on the
470     +         # control-plane, or onto the L4 on a default-local box wired for cloud).
471     +         if (
472     +             resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
473     +             and _cloud_available(cfg)
474     +         ):
475     +             compute = get_compute_backend(cfg, target_override="cloud_run")

```

```

468     +         else:
469     +             compute = None
470     +         if compute is None:
471     -             return None # default-OFF ? run the in-process segmenter below (unchanged)
472     +             return None # default-OFF / placed on control_plane ? the in-process segmenter
below
473
474         storage_cfg = getattr(cfg, "storage", None)
475
476     @@ -623,12 +658,20 @@ def _maybe_dispatch_remote(
477         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
478         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
479         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
480     +     # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
481     +     # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
482     +     # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
483     +     # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators
? see
484     +     # _execute_processing), so the worker must NOT skip: it becomes the gate.
485     +     worker_skips_validation = (
486     +         resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
487     +     )
488     +     spec = ComputeJobSpec(
489     +         video_uri=req.video_path,
490     +         out_uri=out_root,
491     +         segmenter=seg_name,
492     +         config_overrides=tuple(overrides),
493     -         skip_validation=True,
494     +         skip_validation=worker_skips_validation,
495     +     )
496     +     try:
497     +         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
498     @@ -811,12 +854,33 @@ def _execute_processing(
499         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below
(mirrors how it bypasses
500         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
501         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
502     -         val_fingerprint = registry.config_fingerprint(config)
503     +         #
504     +         # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
505     +         # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
506     +         # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
507     +         # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
508     +         # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed
job. When detection
509     +         # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
510     +         # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
511     +         validate_on_control_plane = (
512     +             resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
513     +             or not _will_detect_on_cloud(config, getattr(req, "compute", None))
514     +         )
515     +         if not validate_on_control_plane:

```

```

516 +         logger.info(
517 +             "[API] validation delegated to the L4 worker "
518 +             "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
519 +             "validator suite; the worker is the gate for %s.",
520 +             req.video_path,
521 +         )
522 +         val_fingerprint = (
523 +             registry.config_fingerprint(config) if validate_on_control_plane else ""
524 +         )
525         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
526         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
527         # failing the job. The cache is a pure speedup ? never a new failure mode.
528         reused_validation = False
529 -         if not req.force:
530 +         if validate_on_control_plane and not req.force:
531             try:
532                 cached = db.get_validation_cache(video_id)
533                 if cached is not None and cached.get("fingerprint") == val_fingerprint:
534 @@ -841,11 +905,11 @@ def _execute_processing(
535                 video_id,
536                 exc_info=True,
537             )
538 -         if not reused_validation:
539 +         if validate_on_control_plane and not reused_validation:
540             logger.info(f"Running validations for {req.video_path}...")
541             val_results, val_context = registry.run_all_with_context(local_video,
config)
542             failures = [r for r in val_results if not r.passed]
543 -         if not reused_validation:
544 +         if validate_on_control_plane and not reused_validation:
545             # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
546             # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
547             db.set_validation_cache(
548 @@ -1114,28 +1178,34 @@ def signed_reel_url(cfg, video_id: str, name: str) ->
Optional[str]:
549             return None
550
551
552 -def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
553 -    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
554 -    and return the {status, filepath, download_url} result the SPA polls for. Returns a
555 -    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
556 -    so the SPA always gets a clean verdict instead of a worker-crash error."""
557 -    import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
558 +def _compile_wait_message(elapsed_sec: float) -> str:
559 +    """The live job.progress line while an L4 compile/stitch is in flight (#521;
mirrors
560 +    ``_wait_progress_message``). Minute-granular; under a minute reads as cold-
start."""
561 +    minutes = int(elapsed_sec) // 60
562 +    if minutes < 1:
563 +        return "stitching on cloud GPU (NVENC) ? starting up"
564 +    return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
565
566 -    notify = progress or (lambda stage: None)
567 -    params = json.loads(job.get("params") or "{}")
568 -    video_id = job.get("video_id") or ""

```

```

569 - segment_ids = params.get("segment_ids") or []
570 - locale = params.get("locale")
571
572 - notify("resolving")
573 - try:
574 -     video, kept, out_name = _resolve_compile(
575 -         db, cfg, video_id, segment_ids, params.get("output_filename", "")
576 -     )
577 - except _CompileError as e:
578 -     return {"status": "failed", "message": e.detail, "video_id": video_id}
579 +def _stitch_reel(
580 +    cfg,
581 +    stitching,
582 +    video_id: str,
583 +    source_ref: str,
584 +    time_pairs: List[Tuple[float, float]],
585 +    out_name: str,
586 +    locale: Optional[str],
587 +    notify,
588 +) -> Dict[str, Any]:
589 +    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
590 +    (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
591 +    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
592 +    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the
source, runs the
593 +    trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
594 +    returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
595 +    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
596 +    import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
597
598 - stitching = getattr(cfg, "stitching", None) or StitchingConfig()
599 - output_dir = (
600 -     getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
601 - )
602 @@ -1147,8 +1217,8 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, A
603     begin_padding=stitching.begin_padding,
604     end_padding=stitching.end_padding,
605     watermark=localized_watermark,
606 +    encoder=getattr(stitching, "encoder", "libx264"),
607 )
608 - time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
609
610     notify("stitching")
611     local_src = ""
612 @@ -1157,7 +1227,7 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, A
613     # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
614     # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
615     local_src = storage.acquire_local_input(
616 -        video["filepath"], getattr(cfg, "storage", None)
617 +        source_ref, getattr(cfg, "storage", None)
618 )
619     # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
620     # dispatch) doesn't run multiple compiles at once on the box (see

```

```

_LOCAL_COMPUTE_LANE).
621     @@ -1175,13 +1245,14 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
622         finally:
623             if local_src:
624                 storage.cleanup_acquired_local_input(
625                     - video["filepath"], local_src, getattr(cfg, "storage", None)
626                     + source_ref, local_src, getattr(cfg, "storage", None)
627                 )
628
629             # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
630             # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
631             # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
632             - # still falls back to the local file on the same instance).
633             + # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
634             + # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
635             reel_uri = reel_object_uri(cfg, video_id, out_name)
636             if reel_uri:
637                 try:
638     @@ -1201,4 +1272,189 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
639             "filepath": out_path,
640             "download_url": download_url,
641             "video_id": video_id,
642             + "reel_uri": reel_uri or "",
643         }
644     +
645     +
646     +def _dispatch_compile_remote(
647     +    cfg,
648     +    db,
649     +    video_id: str,
650     +    segment_ids: List[int],
651     +    output_filename: str,
652     +    locale: Optional[str],
653     +    notify,
654     +) -> Dict[str, Any]:
655     +    """"#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
656     +    control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
657     +    small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
658     +    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
659     +    done-marker, and returns the same ``{status, filepath, download_url}`` shape the
SPA expects. The
660     +    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
661     +    ``_maybe_dispatch_remote`` (the DETECT dispatch).""""
662     +    from backend.compute import CompileJobSpec, get_compute_backend
663     +
664     +    notify("resolving")
665     +    try:
666     +        video, kept, out_name = _resolve_compile(
667     +            db, cfg, video_id, segment_ids, output_filename
668     +        )
669     +    except _CompileError as e:
670     +        return {"status": "failed", "message": e.detail, "video_id": video_id}
671     +
672     +    stitching = getattr(cfg, "stitching", None) or StitchingConfig()

```

```

673 +     storage_cfg = getattr(cfg, "storage", None)
674 +     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
675 +     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
676 +     if compute is None:
677 +         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
678 +         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail
? a slow reel
679 +         # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
680 +         logger.warning(
681 +             "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
682 +             "stitching in-process on the control-plane instead.",
683 +             out_root,
684 +         )
685 +         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
686 +         return _stitch_reel(
687 +             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
688 +         )
689 +
690 +     # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-
safe on the wire);
691 +     # the stitching config is the control-plane's RESOLVED one, with the encoder
swapped to the L4's
692 +     # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-
encodes the reel.
693 +     stitch_dump = stitching.model_dump()
694 +     stitch_dump["encoder"] = getattr(
695 +         getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
696 +     )
697 +     spec_payload = {
698 +         "video_id": video_id,
699 +         "video_uri": video["filepath"],
700 +         "output_name": out_name,
701 +         "locale": locale,
702 +         "time_pairs": [
703 +             [float(s["start_time"]), float(s["end_time"])] for s in kept
704 +         ],
705 +         "stitching": stitch_dump,
706 +     }
707 +     token = uuid.uuid4().hex
708 +     spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
709 +     try:
710 +         notify("staging compile spec")
711 +         with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
712 +             local_spec = os.path.join(td, "spec.json")
713 +             with open(local_spec, "w", encoding="utf-8") as f:
714 +                 json.dump(spec_payload, f)
715 +                 storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
716 +     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
717 +         logger.error(
718 +             "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
719 +         )
720 +         return {
721 +             "status": "failed",
722 +             "message": f"cloud compile: could not stage the compile spec: {e}",
723 +             "video_id": video_id,
724 +         }
725 +
726 +     # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the

```

```

spec, mirror the
727 + # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
728 + overrides = [f"storage.output_root={out_root}"]
729 + backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
730 + if backend_name:
731 +     overrides.append(f"storage.backend={backend_name}")
732 + spec = CompileJobSpec(
733 +     out_uri=out_root,
734 +     spec_uri=spec_uri,
735 +     video_id=video_id,
736 +     output_name=out_name,
737 +     config_overrides=tuple(overrides),
738 + )
739 +
740 + notify("dispatching stitch (cloud_run)")
741 + logger.info(
742 +     "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
743 +     video_id,
744 +     out_root,
745 +     len(kept),
746 + )
747 + try:
748 +     result = compute.run_compile(
749 +         spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
750 +     )
751 + except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
752 +     logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
753 +     return {
754 +         "status": "failed",
755 +         "message": f"cloud compile dispatch error: {e}",
756 +         "video_id": video_id,
757 +     }
758 + if result.returncode != 0:
759 +     msg = str(
760 +         result.report.get("message")
761 +         or f"cloud compile job failed (returncode {result.returncode})."
762 +     )
763 +     return {"status": "failed", "message": msg, "video_id": video_id}
764 +
765 + download_url = str(
766 +     result.report.get("download_url")
767 +     or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
768 + )
769 + logger.info(
770 +     "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
771 +     result.execution,
772 +     result.outputs_uri,
773 +     video_id,
774 + )
775 + notify("finished")
776 + return {
777 +     "status": "success",
778 +     "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
779 +     "download_url": download_url,
780 +     "video_id": video_id,
781 +     "compute": {
782 +         "path": "cloud_run",
783 +         "execution": result.execution,
784 +         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
785 +         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
786 +     },
787 + }

```

```

788     +
789     +
790     +def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
791     +     """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
792     +     the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
793     +     result for an EXPECTED problem (missing video / no segments / a stitch error) so
the SPA always gets
794     +     a clean verdict instead of a worker-crash error.
795     +
796     +     #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
797     +     dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
798     +     (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
799     +     2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
800     +     notify = progress or (lambda stage: None)
801     +     params = json.loads(job.get("params") or "{}")
802     +     video_id = job.get("video_id") or ""
803     +     segment_ids = params.get("segment_ids") or []
804     +     locale = params.get("locale")
805     +     output_filename = params.get("output_filename", "")
806     +
807     +     if (
808     +         resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
809     +         and _cloud_available(cfg)
810     +     ):
811     +         return _dispatch_compile_remote(
812     +             cfg, db, video_id, segment_ids, output_filename, locale, notify
813     +         )
814     +
815     +     notify("resolving")
816     +     try:
817     +         video, kept, out_name = _resolve_compile(
818     +             db, cfg, video_id, segment_ids, output_filename
819     +         )
820     +     except _CompileError as e:
821     +         return {"status": "failed", "message": e.detail, "video_id": video_id}
822     +
823     +     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
824     +     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
825     +     return _stitch_reel(
826     +         cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
827     +     )
828     diff --git a/backend/pipeline/stitcher/compiler.py
b/backend/pipeline/stitcher/compiler.py
829     index 65d1c57..5c7229e 100644
830     --- a/backend/pipeline/stitcher/compiler.py
831     +++ b/backend/pipeline/stitcher/compiler.py
832     @@ -23,6 +23,49 @@ _BUNDLED_FONT = (
833     +     Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
834     + )
835
836     +#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
837     +#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
838     +_ENCODER_AVAILABLE: Dict[str, bool] = {}
839     +
840     +
841     +def _probe_encoder_available(encoder: str) -> bool:
842     +     """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny

```

```

1-frame
843 +     lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
844 +     real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
845 +     runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
846 +     if encoder in _ENCODER_AVAILABLE:
847 +         return _ENCODER_AVAILABLE[encoder]
848 +     ok = False
849 +     # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the
configured ffmpeg
850 +     # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
851 +     configured = ffmpeg_timeout_sec()
852 +     probe_timeout = min(30.0, float(configured)) if configured else 30.0
853 +     try:
854 +         proc = subprocess.run(
855 +             [
856 +                 ffmpeg_bin(),
857 +                 "-hide_banner",
858 +                 "-f",
859 +                 "lavfi",
860 +                 "-i",
861 +                 "color=c=black:s=64x64:d=0.1",
862 +                 "-frames:v",
863 +                 "1",
864 +                 "-c:v",
865 +                 encoder,
866 +                 "-f",
867 +                 "null",
868 +                 "-",
869 +             ],
870 +             capture_output=True,
871 +             timeout=probe_timeout,
872 +         )
873 +         ok = proc.returncode == 0
874 +     except Exception: # noqa: BLE001 ? a probe failure/timeout just means "not usable
? fall back"
875 +         ok = False
876 +         _ENCODER_AVAILABLE[encoder] = ok
877 +         return ok
878 +
879
880     class VideoCompiler:
881         def __init__(
882 @@ -31,6 +74,7 @@ class VideoCompiler:
883             end_padding: float = 1.0,
884             begin_padding: float = 0.5,
885             watermark: Optional[Any] = None,
886 +             encoder: str = "libx264",
887         ):
888             self.output_dir = output_dir
889             self.end_padding = end_padding
890 @@ -38,6 +82,11 @@ class VideoCompiler:
891             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
892             # normalized to a dict (or None) so this module stays config-package-agnostic.
893             self.watermark = self._normalize_watermark(watermark)
894 +             # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
895 +             # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
896 +             # re-encode the watermark forces. The requested encoder is PROBED once per run
897 +             # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run
it.
898 +             self.encoder = (encoder or "libx264").strip() or "libx264"

```

```

899         os.makedirs(output_dir, exist_ok=True)
900
901     @staticmethod
902     @@ -225,6 +274,30 @@ class VideoCompiler:
903         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
904         return drawtext
905
906     + def _select_encoder(self) -> Tuple[List[str], str]:
907     +     """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
908     +     chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec
and the final
909     +     concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-
encode; if the
910     +     requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to
911     +     libx264 so a misconfig never nukes the reel."""
912     +     enc = self.encoder
913     +     if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
914     +         logger.warning(
915     +             "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
916     +             "libx264 (CPU). On the L4 worker this should be available.",
917     +             enc,
918     +         )
919     +     enc = "libx264"
920     +     if enc == "libx264":
921     +         # Unchanged historical CPU settings (byte-compatible with the pre-#521
stitch).
922     +         return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
923     +         # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
924     +         # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
925     +         return (
926     +             ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
927     +             enc,
928     +         )
929     +
930     def _trim_one_segment(
931         self,
932         idx: int,
933     @@ -235,6 +308,7 @@ class VideoCompiler:
934         tmp_dir: str,
935         raw_video_path: str,
936         watermark_filter: Optional[str],
937     +     video_codec_args: Optional[List[str]] = None,
938     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
939         """Trim a single segment into its own clip file (extracted verbatim from the
940         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
941     @@ -297,6 +371,13 @@ class VideoCompiler:
942         f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
943         )
944
945     +     # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
946     +     # directly without resolving an encoder ? keeps this method's old behaviour
intact.
947     +     codec_args = (
948     +         video_codec_args
949     +         if video_codec_args is not None
950     +         else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
951     +     )
952     +     base_cmd = [
953         ffmpeg_bin(),

```

```

954         "-y",
955     @@ -306,12 +387,7 @@ class VideoCompiler:
956         str(round(padded_end, 3)),
957         "-i",
958         raw_video_path,
959     -         "-c:v",
960     -         "libx264",
961     -         "-preset",
962     -         "superfast",
963     -         "-crf",
964     -         "22",
965     +         *codec_args,
966         "-c:a",
967         "aac",
968         "-b:a",
969     @@ -445,6 +521,11 @@ class VideoCompiler:
970         watermark_filter = self._watermark_filter(tmp_dir)
971         if watermark_filter:
972             logger.info("[watermark] applying branding overlay to each clip.")
973     +         # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
974     +         # and the final concat stays -c copy (#521).
975     +         video_codec_args, chosen_encoder = self._select_encoder()
976     +         if chosen_encoder != "libx264":
977     +             logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
978             for idx, (raw_start, raw_end) in enumerate(segments):
979                 clip_path, skip_record = self._trim_one_segment(
980                     idx,
981             @@ -455,6 +536,7 @@ class VideoCompiler:
982                 tmp_dir,
983                 raw_video_path,
984                 watermark_filter,
985             +                 video_codec_args,
986                 )
987             if clip_path is not None:
988                 temp_clips.append(clip_path)
989 diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
990 index 6d3420b..79db808 100644
991 --- a/backend/worker/__main__.py
992 +++ b/backend/worker/__main__.py
993     @@ -419,6 +419,108 @@ def cmd_infer(args: argparse.Namespace) -> int:
994         return 0
995
996
997     +def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
998     +    """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
999     +    resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
1000     +    A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
1001     +    local = storage.fetch(
1002     +        spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
1003     +    )
1004     +    with open(local, encoding="utf-8") as f:
1005     +        data = json.load(f)
1006     +    if not isinstance(data, dict):
1007     +        raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
1008     +    return data
1009     +
1010     +
1011     +def _write_compile_marker(
1012     +    done_uri: str,
1013     +    result: dict,

```

```

1014 +     video_id: str,
1015 +     returncode: int,
1016 +     storage_cfg: dict,
1017 +     scratch: str,
1018 + ) -> None:
1019 +     """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
1020 +     dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
1021 +     try:
1022 +         marker = {
1023 +             "video_id": video_id,
1024 +             "returncode": returncode,
1025 +             "status": result.get("status", "failed"),
1026 +             "download_url": result.get("download_url", ""),
1027 +             "reel_uri": result.get("reel_uri", ""),
1028 +             "message": result.get("message", ""),
1029 +         }
1030 +         local = os.path.join(scratch, "_compile_done.json")
1031 +         with open(local, "w", encoding="utf-8") as f:
1032 +             json.dump(marker, f)
1033 +         storage.put(local, done_uri, config=storage_cfg)
1034 +         logger.info("[worker] wrote compile marker -> %s", done_uri)
1035 +     except Exception as e: # never fail a good stitch because the marker push hiccuped
1036 +         logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
1037 +
1038 +
1039 + def cmd_compile(args: argparse.Namespace) -> int:
1040 +     """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU).
The stitch is the
1041 +     2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
1042 +     the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
1043 +     the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses
(so there is no
1044 +     forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
1045 +     bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
1046 +     from backend import orchestration
1047 +     from backend.config import StitchingConfig
1048 +
1049 +     config = load_config(args.config) if args.config else load_config()
1050 +     _apply_overrides(config, args.set)
1051 +     storage_cfg = config.storage.model_dump()
1052 +
1053 +     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
1054 +     os.makedirs(scratch, exist_ok=True)
1055 +     # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
1056 +     config.output.output_dir = scratch
1057 +     # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
1058 +     # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
1059 +     # destination even if the override were dropped.
1060 +     if not (config.storage.output_root or "").strip():
1061 +         config.storage.output_root = args.out_uri

```

12. assistant (2026-07-08T06:05:01.702Z)

Let me read the rest of the diff (worker cmd_compile and tests).

13. user (2026-07-08T06:05:03.827Z)

```
1062 +
1063 +     spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
1064 +     video_id = str(spec.get("video_id") or "")
1065 +     out_name = str(spec.get("output_name") or "")
1066 +     source_ref = str(spec.get("video_uri") or "")
1067 +     locale = spec.get("locale")
1068 +     time_pairs = [
1069 +         (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
1070 +     ]
1071 +     stitching = StitchingConfig(**(spec.get("stitching") or {}))
1072 +
1073 +     print(
1074 +         f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
1075 +         f"encoder={stitching.encoder} out={out_name}"
1076 +     )
1077 +
1078 +     result = orchestration._stitch_reel(
1079 +         config,
1080 +         stitching,
1081 +         video_id,
1082 +         source_ref,
1083 +         time_pairs,
1084 +         out_name,
1085 +         locale,
1086 +         lambda stage: print(f"[worker] compile: {stage}"),
1087 +     )
1088 +     status = str(result.get("status") or "failed")
1089 +     rc = 0 if status == "success" else 3
1090 +     if status != "success":
1091 +         print(f"[worker] compile FAILED: {result.get('message')}")
1092 +
1093 +     # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
1094 +     # passes it.
1095 +     if getattr(args, "done_uri", None):
1096 +         _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg,
1097 +                               scratch)
1098 +
1099 +     return rc
1100 +
1101 + #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
1102 + #: so a
1103 + #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
1104 + #: collision).
1105 + _VIDEO_EXTS = ("mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
1106 + @ -760,6 +862,43 @@ def build_parser() -> argparse.ArgumentParser:
1107 +     )
1108 +     pi.set_defaults(func=cmd_infer)
1109 +
1110 +     pc = sub.add_parser(
1111 +         "compile",
1112 +         help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
1113 +         compile-spec",
1114 +     )
1115 +     pc.add_argument(
1116 +         "--spec-uri",
1117 +         required=True,
1118 +         help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
1119 +         stitching "
1120 +         "config + source uri + video_id)",
1121 +     )
1122 +     pc.add_argument(
1123 +         "--out-uri",
1124 +         default="",
```

```

1119 +         help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
1120 +         "Also forwarded via --set storage.output_root; either resolves the reel path.",
1121 +     )
1122 +     pc.add_argument(
1123 +         "--done-uri",
1124 +         default=None,
1125 +         help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
1126 +         "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
1127 +     )
1128 +     pc.add_argument(
1129 +         "--config", default=None, help="config.json path (default: ./config.json)"
1130 +     )
1131 +     pc.add_argument(
1132 +         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
1133 +     )
1134 +     pc.add_argument(
1135 +         "--set",
1136 +         action="append",
1137 +         default=[],
1138 +         metavar="k=v",
1139 +         help="config override, e.g. --set storage.output_root=gs://bucket",
1140 +     )
1141 +     pc.set_defaults(func=cmd_compile)
1142 +
1143     pt = sub.add_parser(
1144         "train",
1145         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
1146         diff --git a/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
b/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
1147     new file mode 100644
1148     index 0000000..d0f7922
1149     --- /dev/null
1150     +++ b/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
1151     @@ -0,0 +1,169 @@
1152     +# Phase ? machine placement + compile-on-L4 (#521)
1153     +
1154     +> **Status:** ? **SHIPPED (code)** . **Owner:** Avi . **Created:** 2026-07-08 .
**Issue:** [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521)
1155     +> **Relation:** extends [COMPUTE_DECOUPLED_SERVING](./README.md) (the `ComputeBackend`
dispatch seam) and **refines D16(b)** (NVENC-encode-deferred-behind-a-config-knob) now that the
2026-07-08 live CUJ supplies the cost data D16(b) said to revisit on.
1156     +> **Evidence:** `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md`
$9 (2026-07-08 live CUJ, rev `rally-control-plane-00019`).
1157     +
1158     +---
1159     +
1160     +## 0. TL;DR
1161     +
1162     +The compile/stitch phase used to run **in-process on the CPU-only 2-vCPU control-
plane**, where the
1163     +2026-07-08 live CUJ measured **~12m55s** for an 11-clip / ~80 s reel from a 1.83 GiB 4K
source
1164     +(~45?90 s per ~7 s clip ? the per-clip branding watermark forces a full 4K re-encode
with **no
1165     +NVENC**). Validation (183 s) runs there too; both serialize on `_LOCAL_COMPUTE_LANE`,
so one run pins
1166     +the control-plane for ~16 min ? the API can't scale past a handful of concurrent users.
1167     +
1168     +#521 makes **where each pipeline phase runs a config edit, not a code change**, and
moves the stitch
1169     +to the **L4 worker (NVENC + 8 vCPU)** the way detection already dispatches:
1170     +

```

```

1171 +1. **`deployment.phase_placement`** ? a per-phase knob (`validation` / `segment` /
`compile`) with
1172 + values `control_plane | cloud_run_l4`. Unset ? **today's placement, byte-
identical**.
1173 +2. **compile ? L4** ? a `kind='compile'` Cloud Run dispatch mirroring the detect path:
the
1174 + control-plane resolves the reel's segments, stages a compile-spec to the bucket,
dispatches
1175 + `python -m backend.worker compile` on the L4, bucket-polls a done-marker, and
returns the same
1176 + `{status, filepath, download_url}` shape the SPA already expects.
1177 +3. **NVENC on the L4** ? `deployment.cloud_stitch_encoder` (default `h264_nvenc`) is
forwarded onto
1178 + the worker's `stitching.encoder`; libx264 stays the everywhere-safe fallback (auto-
probed).
1179 +
1180 +The cloud-serving preset opts compile onto the L4
(`phase_placement.compile=cloud_run_l4`).
1181 +
1182 +---
1183 +
1184 +## 1. The placement model
1185 +
1186 +`resolve_phase_placement(cfg, phase)` (in `backend/config/placement.py`) is the
**single source of
1187 +truth** ? orchestration and the worker never re-derive the mapping. Precedence: an
explicit
1188 +`deployment.phase_placement.<phase>` wins; otherwise the **legacy default** reproduces
pre-#521
1189 +behaviour exactly:
1190 +
1191 +| phase | knob unset (legacy default) | meaning |
1192 +|---|---|---|
1193 +| `validation` | `control_plane` | the authoritative gate (#512/#513) stays on the
control-plane |
1194 +| `segment` (detect) | `cloud_run_l4` **iff** `compute_target==cloud_run`, else
`control_plane` | today's DETECT dispatch |
1195 +| `compile` (stitch) | `control_plane` | the in-process ffmpeg stitch |
1196 +
1197 +So a config with **no** `phase_placement` block behaves exactly as before this existed.
To relocate a
1198 +phase, set its knob:
1199 +
1200 +```jsonc
1201 +// Move the stitch to the L4 (what the cloud-serving preset does):
1202 +{ "deployment": { "compute_target": "cloud_run",
1203 +                  "phase_placement": { "compile": "cloud_run_l4" } } }
1204 +
1205 +// Force detection back onto the control-plane on a cloud box (e.g. a debugging run):
1206 +{ "deployment": { "compute_target": "cloud_run",
1207 +                  "phase_placement": { "segment": "control_plane" } } }
1208 +
1209 +// Move validation off the control-plane too (must pair with a cloud segment ? see §4):
1210 +{ "deployment": { "compute_target": "cloud_run",
1211 +                  "phase_placement": { "validation": "cloud_run_l4" } } }
1212 +```
1213 +
1214 +`control_plane` runs the phase in-process on the (CPU-only) control-plane service;
`cloud_run_l4`
1215 +dispatches it to the GPU/NVENC Cloud Run worker Job.
1216 +
1217 +---
1218 +
1219 +## 2. compile ? L4 dispatch (the flow)
1220 +

```

```

1221 +`orchestration._execute_compile` consults `resolve_phase_placement(cfg, "compile")`.
When it resolves
1222 +to `cloud_run_l4` **and** cloud dispatch is available (`_cloud_available`), it calls
1223 +`_dispatch_compile_remote`; otherwise it stitches in-process via the shared
`_stitch_reel` core
1224 +(unchanged).
1225 +
1226 +`_dispatch_compile_remote` (mirrors `_maybe_dispatch_remote`):
1227 +
1228 +1. **Resolve** the reel's segments HERE ? the control-plane owns the DB
(`_resolve_compile` ?
1229 + `(video, kept, out_name)`), applying the `stitching.min_shot_count` floor as before.
1230 +2. **Stage a compile-spec** to `**no control-plane DB** ? everything rides the spec.
1234 +3. **Dispatch** a `compile` Cloud Run Job execution (`CompileJobSpec` ?
`CloudRunCompute.run_compile`)
1235 + running `python -m backend.worker compile --spec-uri ? --done-uri ? --set
storage.output_root=?`,
1236 + forwarding the bucket coordinates (the worker Job has no `DEPLOYMENT_PROFILE`).
1237 +4. **Bucket-poll** the done-marker at `**shared
`_stitch_reel`
1242 + core** (no forked pipeline), mirror-uploads it to
`<output_root>/highlights/<video_id>/<name>`,
1243 + and writes the marker (rc + `download_url` + `reel_uri`).
1244 +6. The control-plane returns `{status, filepath, download_url, video_id}` ? the reel is
already in the
1245 + bucket, so `/api/highlights` serves it via the existing signed-URL 307. On a worker
rc?0 ?
1246 + `status='failed'` with the worker's message (clean SPA verdict, never a crash).
1247 +
1248 +**Fallback:** if compile is placed on the L4 but the box isn't actually cloud-wired (no
output bucket
1249 +/ Cloud Run env), `_dispatch_compile_remote` logs and stitches in-process ? a slow reel
beats no reel.
1250 +
1251 +---
1252 +
1253 +## 3. NVENC on the L4 ? and the D5/D16(b) parity note
1254 +
1255 +The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding
watermark forces
1256 +(~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521
adds an encoder
1257 +knob:
1258 +
1259 +- `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) |
`h264_nvenc` |
1260 + `hevc_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a
1-frame lavfi test
1261 + encode) and **falls back to libx264** if this ffmpeg build can't run it, so a
misconfig / non-GPU
1262 + box never breaks. All clips share the one chosen encoder ? the concat stays `-c
copy`.
1263 +- `deployment.cloud_stitch_encoder` (default `h264_nvenc`) ? what the compile-dispatch

```

```

forwards onto
1264 + the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4
stitch on CPU.
1265 +- The control-plane's own local stitch always uses `stitching.encoder` (libx264 by
default) ? the
1266 + CP has no GPU.
1267 +
1268 Parity note (honest): [README.md](./README.md) D5/D16(b) kept stitch encode =
libx264 to
1269 +protect a byte-exact stitch parity guarantee, and explicitly deferred NVENC encode
behind a config
1270 +knob, "revisit only if M7 cost data justifies relaxing parity."The 2026-07-08 CUJ
(~13 min stitch,
1271 +a hard scaling wall past ~5 concurrent users) is that cost data. #521 lands exactly
the deferred
1272 +shape ? NVENC behind a config knob, libx264 the default, config-reversible per
phase. Enabling
1273 +NVENC relaxes byte-identity at the reel ENCODE layer only (a cosmetic, shareable
artifact); the
1274 WINDOW/rally-level parity (the analytical output) is untouched ? detection and
windowing are
1275 +unchanged. If bit-exact reels are ever required, set `cloud_stitch_encoder=libx264`.
1276 +
1277 +> Follow-up (tracked, not in #521's first cut): a pre-rendered / overlay-filter
watermark so the
1278 +> stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).
1279 +
1280 +---
1281 +
1282 ## 4. Validation placement + the "nobody validates" guard
1283 +
1284 +`phase_placement.validation` moves the CPU-bound validator suite off the 2-vCPU
control-plane too,
1285 +via the existing `--skip-validation` lever (#513):
1286 +
1287 +- control_plane (default): the control-plane runs the validators and is the
authoritative gate;
1288 + the worker skips its redundant re-validation (`skip_validation=True`) ? today's
#512/#513
1289 + behaviour, byte-identical.
1290 +- cloud_run_l4: when detection actually dispatches to the L4, the control-
plane delegates
1291 + ? it does NOT run the validator suite (`execute_processing` skips it), and the
dispatch carries
1292 + no `--skip-validation`, so the worker validates and rejects a bad clip via a
non-zero rc
1293 + (surfaced as a failed job).
1294 +
1295 Footgun guard: a clip can only be validated on the worker if the worker also
detects it there.
1296 +Two protections:
1297 +
1298 +1. Config-load guard (`DeploymentConfig` model-validator):
`validation=cloud_run_l4` requires the
1299 + effective segment placement to also be `cloud_run_l4` (explicit, or via
`compute_target=cloud_run`)
1300 + ? else config load fails with an actionable error.
1301 +2. Runtime guard (`will_detect_on_cloud`): if a per-request `compute='local'`
forces detection
1302 + in-process (no worker), the control-plane still validates ? never leave a clip
unvalidated
1303 + because nobody ran the gate.
1304 +
1305 +---
1306 +

```

```

1307   +## 5. Files & tests
1308   +
1309   +| Area | Code | Tests |
1310   +|---|---|---|
1311   +| Placement resolver + config | `backend/config/placement.py`,
`backend/config/models.py` (`PhasePlacementConfig`, `DeploymentConfig.phase_placement` /
`cloud_stitch_encoder`, guard) | `tests/test_phase_placement.py`,
`tests/test_config_deployment.py` |
1312   +| Preset | `backend/config/presets.py` (`cloud-serving` ?
`phase_placement.compile=cloud_run_l4`) | `tests/test_phase_placement.py` |
1313   +| compile dispatch | `backend/orchestration.py` (`_execute_compile`,
`_dispatch_compile_remote`, `_stitch_reel`, `_will_detect_on_cloud`) |
`tests/test_compile_dispatch.py` |
1314   +| compute backend | `backend/compute/base.py` (`CompileJobSpec`, `run_compile`),
`backend/compute/cloud_run.py` (`run_compile`, `_await_marker`) |
`tests/test_compute_backend.py` |
1315   +| worker | `backend/worker/__main__.py` (`compile` subcommand, `cmd_compile`) |
`tests/test_worker.py` |
1316   +| NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`,
`_select_encoder`, `_probe_encoder_available`) | `tests/test_compiler.py` |
1317   +| validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip
flag, `_execute_processing`) | `tests/test_api_cloud_dispatch.py` |
1318   +
1319   +All GPU-free + cloud-free: a fake `CloudRunJobClient` simulates the worker (reads the
staged spec,
1320   +writes the reel + done-marker into a local "bucket"), mirroring
`tests/test_api_cloud_dispatch.py`.
1321   diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md
b/docs/COMPUTE_DECOUPLED_SERVING/README.md
1322   index e60d644..e17cf71 100644
1323   --- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
1324   +++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
1325   @@ -6,6 +6,7 @@
1326   > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the truly-local profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
1327   > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
compute-decoupled serving because truly-local is a co-equal, first-class profile, not an
afterthought.
1328   > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
1329   +> #521 (2026-07-08): phase?machine placement is now config-driven and
compile/stitch is relocatable to the L4 worker (NVENC) ? see
[PHASE_PLACEMENT.md](../PHASE_PLACEMENT.md). This refines D16(b) (NVENC-encode deferred
behind a config knob) now that the live CUJ (§9 of the validation-latency runlog) supplies the
cost data D16(b) said to revisit on.
1330
1331   ---
1332
1333   diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
1334   index 5baaf94..d0b8604 100644
1335   --- a/tests/test_api_cloud_dispatch.py
1336   +++ b/tests/test_api_cloud_dispatch.py
1337   @@ -1103,3 +1103,73 @@ def test_wait_progress_message_is_honest_and_minute_granular():
1338       assert "wasb_hybrid" in early and "starting up" in early
1339       later = _wait_progress_message("wasb_hybrid", 125.0)
1340       assert later.endswith("2 min elapsed")
1341   +
1342   +
1343   +## --- #521: phase_placement.validation (delegate validation to the L4 worker)
-----
1344   +
1345   +

```

```

1346 +def test_validation_delegated_to_worker_when_placed_on_l4(cloud_env):
1347 +     """With phase_placement.validation=cloud_run_l4 (+ segment on the L4), the control-
plane does NOT
1348 +     run its validator suite ? it delegates to the worker, which validates (dispatch
carries NO
1349 +     --skip-validation). This is the #521 scaling move: get the CPU-bound validation off
the 2-vCPU CP."""
1350 +     db, bucket, monkeypatch = cloud_env
1351 +     ran = {"n": 0}
1352 +
1353 +     def _spy(*a, **k):
1354 +         ran["n"] += 1
1355 +         return _pass(), {}
1356 +
1357 +     monkeypatch.setattr(m.registry, "run_all_with_context", _spy)
1358 +     cfg = AppConfig.model_validate(
1359 +         {
1360 +             "deployment": {
1361 +                 "compute_target": "cloud_run",
1362 +                 "phase_placement": {"validation": "cloud_run_l4"},
1363 +             },
1364 +             "storage": {"backend": "local", "output_root": str(bucket)},
1365 +         }
1366 +     )
1367 +     fake = _FakeCloudClient()
1368 +     _inject_fake(monkeypatch, fake)
1369 +
1370 +     out = m._execute_processing(
1371 +         cfg, db, m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
1372 +     )
1373 +     assert out["status"] == "success"
1374 +     assert ran["n"] == 0 # the control-plane skipped its validators (delegated)
1375 +     assert "--skip-validation" not in fake.calls[0] # so the worker validates instead
1376 +
1377 +
1378 +def test_validation_not_delegated_when_detection_runs_in_process(cloud_env,
monkeypatch):
1379 +     """Footgun guard: validation is placed on the L4 but the request forces
compute='local' (detection
1380 +     runs in-process, no worker) ? the control-plane MUST still validate. Never leave a
clip unvalidated
1381 +     because nobody ran the gate."""
1382 +     db, bucket, _mp = cloud_env
1383 +     ran = {"n": 0}
1384 +
1385 +     def _spy(*a, **k):
1386 +         ran["n"] += 1
1387 +         return _pass(), {}
1388 +
1389 +     monkeypatch.setattr(m.registry, "run_all_with_context", _spy)
1390 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
1391 +     cfg = AppConfig.model_validate(
1392 +         {
1393 +             "deployment": {
1394 +                 "compute_target": "cloud_run",
1395 +                 "phase_placement": {"validation": "cloud_run_l4"},
1396 +             },
1397 +             "storage": {"backend": "local", "output_root": str(bucket)},
1398 +         }
1399 +     )
1400 +     fake = _FakeCloudClient()
1401 +     _inject_fake(monkeypatch, fake)
1402 +
1403 +     out = m._execute_processing(

```

```

1404 +         cfg,
1405 +         db,
1406 +         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",
compute="local"),
1407 +     )
1408 +     assert out["status"] == "success"
1409 +     assert ran["n"] == 1 # in-process ? the control-plane validated (no worker to
delegate to)
1410 +     assert fake.calls == [] # ran locally, never dispatched
1411 diff --git a/tests/test_compile_dispatch.py b/tests/test_compile_dispatch.py
1412 new file mode 100644
1413 index 0000000..6bdae1a
1414 --- /dev/null
1415 +++ b/tests/test_compile_dispatch.py
1416 @@ -0,0 +1,204 @@
1417 +"""Compile/stitch ? L4 worker dispatch (#521).
1418 +
1419 +Proves the control-plane can OFFLOAD the reel compile/stitch to the Cloud Run L4 worker
by config
1420 +(``deployment.phase_placement.compile=cloud_run_l4``), mirroring the DETECT dispatch ?
all GPU-free +
1421 +cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads
the staged
1422 +compile-spec + writes the reel + done-marker into a local "bucket"). Cardinal
invariant: DEFAULT
1423 +(compile on the control-plane) still stitches in-process, byte-identically.
1424 +"""
1425 +
1426 +import json
1427 +import os
1428 +
1429 +import pytest
1430 +
1431 +import backend.compute as compute_pkg
1432 +import backend.orchestration as orch
1433 +from backend.compute.cloud_run import CloudRunJobClient
1434 +from backend.config import AppConfig
1435 +from backend.infrastructure.database import Database
1436 +from backend.infrastructure.execution_policy import ExecutionPolicy
1437 +
1438 +_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
1439 +
1440 +
1441 +class _FakeCompileClient(CloudRunJobClient):
1442 +    """Simulates the Cloud Run compile worker: on dispatch, READ the staged compile-
spec (proving the
1443 +    control-plane resolved + staged it), then write the reel done-marker into the local
bucket exactly
1444 +    where the real worker would, so the dispatcher's bucket-poll resolves."""
1445 +
1446 +    def __init__(self, *, returncode=0, video_id=None):
1447 +        self.calls: list[list[str]] = []
1448 +        self.spec: dict | None = None
1449 +        self.returncode = returncode
1450 +        self._video_id = video_id
1451 +
1452 +    def run_job(self, job_name, argv, *, region, project=None):
1453 +        self.calls.append(list(argv))
1454 +        spec_uri = argv[argv.index("--spec-uri") + 1]
1455 +        done_uri = argv[argv.index("--done-uri") + 1]
1456 +        with open(spec_uri, encoding="utf-8") as f:
1457 +            self.spec = json.load(f)
1458 +        vid = self._video_id or self.spec["video_id"]
1459 +        out_name = self.spec["output_name"]
1460 +        status = "success" if self.returncode == 0 else "failed"

```

```
1461 +         marker = {
```

14. user (2026-07-08T06:05:09.211Z)

```
1462 +         "video_id": vid,
1463 +         "returncode": self.returncode,
1464 +         "status": status,
1465 +         "download_url": f"/api/highlights/{vid}/{out_name}",
1466 +         "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
1467 +         "message": "" if self.returncode == 0 else "stitch blew up",
1468 +     }
1469 +     os.makedirs(os.path.dirname(done_uri), exist_ok=True)
1470 +     with open(done_uri, "w", encoding="utf-8") as f:
1471 +         json.dump(marker, f)
1472 +     return "exec-compile-1"
1473 +
1474 + def cancel_execution(self, execution): # pragma: no cover - happy path never
cancels
1475 +     raise AssertionError("cancel must never fire on the happy path")
1476 +
1477 +
1478 +def _inject_fake(monkeypatch, fake):
1479 +     real = compute_pkg.get_compute_backend
1480 +     monkeypatch.setattr(
1481 +         compute_pkg,
1482 +         "get_compute_backend",
1483 +         lambda config, **kw: real(config, run_client=fake, policy=_FAST, token="tok",
**kw),
1484 +     )
1485 +
1486 +
1487 +@pytest.fixture
1488 +def compile_env(tmp_path, monkeypatch):
1489 +     """A control-plane DB with a video + 3 play segments, wired for cloud with compile
placed on the
1490 +     L4 and a LOCAL bucket (so the fake worker's spec/marker are plain file
reads/writes)."""
1491 +     db = Database(str(tmp_path / "t.db"))
1492 +     bucket = tmp_path / "bucket"
1493 +     bucket.mkdir()
1494 +     video_id = "vidCompile"
1495 +     src = tmp_path / "source.mp4"
1496 +     src.write_bytes(b"SRC")
1497 +     db.add_video(video_id, str(src), "processed", [])
1498 +     sid1 = db.add_play_segment(video_id, 2.0, 6.0, metadata={"shot_count": 5})
1499 +     sid2 = db.add_play_segment(video_id, 10.0, 14.0, metadata={"shot_count": 3})
1500 +     sid3 = db.add_play_segment(video_id, 20.0, 22.0, metadata={"shot_count": 4})
1501 +     cfg = AppConfig.model_validate(
1502 +         {
1503 +             "deployment": {
1504 +                 "compute_target": "cloud_run",
1505 +                 "phase_placement": {"compile": "cloud_run_l4"},
1506 +             },
1507 +             "storage": {"backend": "local", "output_root": str(bucket)},
1508 +         }
1509 +     )
1510 +     return db, cfg, video_id, [sid1, sid2, sid3], bucket, monkeypatch
1511 +
1512 +
1513 +def _job(video_id, segment_ids, name="reel.mp4", locale=None):
1514 +     return {
1515 +         "id": "cj1",
1516 +         "video_id": video_id,
1517 +         "kind": "compile",
1518 +         "params": json.dumps(
```

```

1519 +         {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
1520 +     ),
1521 + }
1522 +
1523 +
1524 +def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
1525 +    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
1526 +    fake = _FakeCompileClient()
1527 +    _inject_fake(monkeypatch, fake)
1528 +
1529 +    out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))
1530 +
1531 +    assert out["status"] == "success"
1532 +    assert out["video_id"] == video_id
1533 +    assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
1534 +    assert out["compute"]["path"] == "cloud_run"
1535 +    assert out["compute"]["execution"] == "exec-compile-1"
1536 +    assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
1537 +
1538 +
1539 +def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
1540 +    """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic
1541 +    start/end from the DB)
1542 +    and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so
1543 +    the worker
1544 +    hardware-encodes and needs no control-plane DB."""
1545 +    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
1546 +    fake = _FakeCompileClient()
1547 +    _inject_fake(monkeypatch, fake)
1548 +
1549 +    orch._execute_compile(cfg, db, _job(video_id, seg_ids))
1550 +
1551 +    assert fake.spec is not None
1552 +    assert fake.spec["video_id"] == video_id
1553 +    assert fake.spec["video_uri"].endswith("source.mp4")
1554 +    # 3 resolved (start, end) pairs from the DB rows
1555 +    assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
1556 +    # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
1557 +    assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
1558 +    # the argv is a `compile` subcommand carrying the spec + done marker
1559 +    argv = fake.calls[0]
1560 +    assert argv[0] == "compile"
1561 +    assert "--spec-uri" in argv and "--done-uri" in argv
1562 +    joined = " ".join(argv)
1563 +    assert "storage.output_root=" in joined # bucket forwarded so the worker mirrors
1564 +    the reel
1565 +
1566 +
1567 +def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
1568 +    """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket
1569 +    coordinates
1570 +    (storage.output_root/backend) ? else the worker can't read the spec or mirror the
1571 +    reel."""
1572 +    db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
1573 +    fake = _FakeCompileClient()
1574 +    _inject_fake(monkeypatch, fake)
1575 +
1576 +    orch._execute_compile(cfg, db, _job(video_id, seg_ids))
1577 +    joined = " ".join(fake.calls[0])
1578 +    assert f"storage.output_root={bucket}" in joined
1579 +    assert "storage.backend=local" in joined
1580 +
1581 +
1582 +def test_compile_cloud_failure_returns_failed(compile_env):
1583 +    """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's
1584 +    message ? the SPA

```

```

1578 +     gets a clean verdict, not a crash."""
1579 +     db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
1580 +     fake = _FakeCompileClient(returncode=3)
1581 +     _inject_fake(monkeypatch, fake)
1582 +
1583 +     out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))
1584 +     assert out["status"] == "failed"
1585 +     assert "stitch blew up" in out["message"]
1586 +     assert out["video_id"] == video_id
1587 +
1588 +
1589 +def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
1590 +    """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
1591 +    _stitch_reel
1592 +    and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg
1593 +    runs."""
1594 +    db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
1595 +    # flip compile back to the control-plane (the default) ? everything else stays
1596 +    cloud-wired
1597 +    cfg.deployment.phase_placement.compile = "control_plane"
1598 +    fake = _FakeCompileClient()
1599 +    _inject_fake(monkeypatch, fake)
1600 +    seen = {}
1601 +
1602 +    def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
1603 +        seen["pairs"] = pairs
1604 +        seen["source"] = source
1605 +        return {"status": "success", "video_id": vid, "download_url":
1606 +            f"/api/highlights/{vid}/{name}"}
1607 +
1608 +    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1609 +
1610 +    out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
1611 +    assert out["status"] == "success"
1612 +    assert fake.calls == [] # NEVER dispatched ? the control-plane stitched in-process
1613 +    assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
1614 +    assert seen["source"].endswith("source.mp4")
1615 +
1616 +
1617 +def test_compile_missing_video_fails_before_dispatch(compile_env):
1618 +    """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
1619 +    db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
1620 +    fake = _FakeCompileClient()
1621 +    _inject_fake(monkeypatch, fake)
1622 +    out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
1623 +    assert out["status"] == "failed"
1624 +    assert fake.calls == [] # never dispatched ? resolve failed first
1625 +
1626 +diff --git a/tests/test_compiler.py b/tests/test_compiler.py
1627 +index 1f51622..f3f2ae9 100644
1628 +--- a/tests/test_compiler.py
1629 ++++ b/tests/test_compiler.py
1630 +@@ -158,3 +158,67 @@ def test_compile_raises_when_source_missing(tmp_path):
1631 +     comp = VideoCompiler(str(tmp_path / "out"))
1632 +     with pytest.raises(FileNotFoundError):
1633 +         comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
1634 +
1635 +
1636 +def test_select_encoder_default_is_libx264(tmp_path):
1637 +    """Default encoder = libx264 (CPU) with the historical settings ? no probe, works
1638 +    everywhere."""
1639 +    comp = VideoCompiler(str(tmp_path / "out"))

```

```

1637 +     args, chosen = comp._select_encoder()
1638 +     assert chosen == "libx264"
1639 +     assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
1640 +
1641 +
1642 +def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):
1643 +     """h264_nvenc resolves to the hardware encoder args when the probe reports it
runnable (the L4)."""
1644 +     from backend.pipeline.stitcher import compiler as comp_mod
1645 +
1646 +     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
1647 +     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
1648 +     args, chosen = comp._select_encoder()
1649 +     assert chosen == "h264_nvenc"
1650 +     assert args[:2] == ["-c:v", "h264_nvenc"]
1651 +     assert "-cq" in args # NVENC constant-quality knob (not -crf)
1652 +
1653 +
1654 +def test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path,
monkeypatch):
1655 +     """On a box where NVENC can't actually run (CI / no GPU), the requested encoder
falls back to
1656 +     libx264 so a misconfig never nukes the reel."""
1657 +     from backend.pipeline.stitcher import compiler as comp_mod
1658 +
1659 +     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: False)
1660 +     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
1661 +     args, chosen = comp._select_encoder()
1662 +     assert chosen == "libx264"
1663 +     assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
1664 +
1665 +
1666 +def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
1667 +     """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim
command
1668 +     carries -c:v h264_nvenc, and the concat stays -c copy."""
1669 +     from backend.pipeline.stitcher import compiler as comp_mod
1670 +
1671 +     monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
1672 +     comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
1673 +     raw = tmp_path / "raw.mp4"
1674 +     raw.write_bytes(b"x")
1675 +     trims = []
1676 +
1677 +     def fake_run(cmd, *a, **k):
1678 +         if cmd[0] == "ffprobe":
1679 +             return MagicMock(stdout="100.0\n", returncode=0)
1680 +         out = cmd[-1]
1681 +         if isinstance(out, str) and out.endswith(".mp4"):
1682 +             open(out, "wb").write(b"clip")
1683 +             if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
1684 +                 trims.append(cmd)
1685 +             return MagicMock(returncode=0, stdout=b"", stderr=b"")
1686 +
1687 +     with patch.object(subprocess, "run", side_effect=fake_run):
1688 +         comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
1689 +         assert trims, "no trim commands captured"
1690 +         for cmd in trims:
1691 +             assert "h264_nvenc" in cmd # hardware encode on every clip
1692 +             assert "libx264" not in cmd
1693 +diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
1694 +index 5444a34..121ab6e 100644
1695 +--- a/tests/test_compute_backend.py
1696 ++++ b/tests/test_compute_backend.py
1697 +@@ -1100,3 +1100,114 @@ def

```

```

test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
1698     )
1699     assert seen["on_tick"] is heartbeat
1700     assert result.video_id == "vidCloud"
1701 +
1702 +
1703 +## ----- compile dispatch (#521 ? L4
stitch)
1704 +
1705 +
1706 +class _FakeCompileClient(CloudRunJobClient):
1707 +     """Simulates the Cloud Run COMPILE worker: on dispatch, write a reel done-marker
into the local
1708 +     bucket exactly where the real ``cmd_compile`` would, so ``run_compile``'s bucket-
poll resolves."""
1709 +
1710 +     def __init__(self, *, video_id="vidReel", returncode=0):
1711 +         self.calls: list[dict] = []
1712 +         self.video_id, self.returncode = video_id, returncode
1713 +
1714 +     def run_job(self, job_name, argv, *, region, project=None):
1715 +         self.calls.append({"job": job_name, "argv": list(argv)})
1716 +         done_uri = argv[argv.index("--done-uri") + 1]
1717 +         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
1718 +         with open(done_uri, "w", encoding="utf-8") as f:
1719 +             json.dump(
1720 +                 {
1721 +                     "video_id": self.video_id,
1722 +                     "returncode": self.returncode,
1723 +                     "status": "success" if self.returncode == 0 else "failed",
1724 +                     "download_url": f"/api/highlights/{self.video_id}/reel.mp4",
1725 +                     "reel_uri": f"gs://b/highlights/{self.video_id}/reel.mp4",
1726 +                 },
1727 +                 f,
1728 +             )
1729 +         return "exec-compile"
1730 +
1731 +     def cancel_execution(self, execution): # pragma: no cover - happy path never
cancels
1732 +         raise AssertionError("cancel must never fire on the happy path")
1733 +
1734 +
1735 +def test_run_compile_dispatches_and_returns_marker(tmp_path):
1736 +     """run_compile dispatches a `compile` subcommand (spec + done marker), bucket-
polls, and returns
1737 +     the worker's rc + download_url from the marker."""
1738 +     from backend.compute.base import CompileJobSpec
1739 +
1740 +     client = _FakeCompileClient(video_id="vidReel")
1741 +     backend = CloudRunCompute(
1742 +         run_client=client, job_name="rally-worker", region="r", policy=_FAST,
token="tok"
1743 +     )
1744 +     out = str(tmp_path / "bucket")
1745 +     result = backend.run_compile(
1746 +         CompileJobSpec(
1747 +             out_uri=out,
1748 +             spec_uri=f"{out}/_compile/tok/spec.json",
1749 +             video_id="vidReel",
1750 +             output_name="reel.mp4",
1751 +             config_overrides=("storage.output_root=" + out,),
1752 +         )
1753 +     )
1754 +     assert result.returncode == 0
1755 +     assert result.video_id == "vidReel"

```

```

1756 + assert result.report["download_url"] == "/api/highlights/vidReel/reel.mp4"
1757 + argv = client.calls[0]["argv"]
1758 + assert argv[0] == "compile"
1759 + assert argv[argv.index("--spec-uri") + 1] == f"{out}/_compile/tok/spec.json"
1760 + assert "--done-uri" in argv
1761 + # the CP-forwarded storage override rides as --set (compile carries NO infer
container overrides)
1762 + assert "storage.output_root=" + out in " ".join(argv)
1763 + assert "deployment.compute_target=local_gpu" not in " ".join(argv)
1764 +
1765 +
1766 +def test_run_compile_surfaces_nonzero_returncode(tmp_path):
1767 +     """A worker stitch failure (rc != 0) is returned as a non-zero ComputeResult (the
dispatcher maps
1768 +     it to a failed job) ? never a false success."""
1769 +     from backend.compute.base import CompileJobSpec
1770 +
1771 +     client = _FakeCompileClient(returncode=3)
1772 +     backend = CloudRunCompute(
1773 +         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
1774 +     )
1775 +     out = str(tmp_path / "bk")
1776 +     result = backend.run_compile(
1777 +         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1778 +     )
1779 +     assert result.returncode == 3
1780 +
1781 +
1782 +def test_compile_marker_prefix_distinct_from_infer(tmp_path):
1783 +     """The compile marker lives under _compile/ (not infer's _dispatch/) so a compile +
a detection
1784 +     for the same bucket never collide on a marker."""
1785 +     from backend.compute.base import CompileJobSpec
1786 +
1787 +     client = _FakeCompileClient()
1788 +     backend = CloudRunCompute(
1789 +         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
1790 +     )
1791 +     out = str(tmp_path / "bk")
1792 +     backend.run_compile(
1793 +         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1794 +     )
1795 +     done_uri = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
1796 +     assert "/_compile/" in done_uri and "/_dispatch/" not in done_uri
1797 +
1798 +
1799 +def test_run_compile_default_backend_raises_not_implemented():
1800 +     """The ABC default: a backend that can't offload a stitch raises
NotImplementedError (only
1801 +     CloudRunCompute overrides run_compile) ? never a silent no-op."""
1802 +     from backend.compute.base import CompileJobSpec, ComputeBackend, ComputeResult
1803 +
1804 +     class _InProc(ComputeBackend):
1805 +         def run_infer(self, spec, *, on_wait=None):
1806 +             return ComputeResult(returncode=0)
1807 +
1808 +     with pytest.raises(NotImplementedError):
1809 +         _InProc().run_compile(
1810 +             CompileJobSpec(out_uri="x", spec_uri="y", video_id="v",
output_name="r.mp4")
1811 +         )
1812 + diff --git a/tests/test_config_deployment.py b/tests/test_config_deployment.py
1813 + index 1e4cfa6..5122bcd 100644

```

```

1814 --- a/tests/test_config_deployment.py
1815 +++ b/tests/test_config_deployment.py
1816 @@ -70,6 +70,9 @@ def test_default_config_dump_only_adds_deployment_section():
1817     "compute_target": "",
1818     "max_concurrent_remote_jobs": 4,
1819     "remote_poll_max_wait_sec": 3900.0,
1820 +    # #521: per-phase placement (unset ? legacy default) + the L4 stitch encoder.
1821 +    "phase_placement": {"validation": None, "segment": None, "compile": None},
1822 +    "cloud_stitch_encoder": "h264_nvenc",
1823 }
1824
1825
1826 @@ -397,6 +400,8 @@ def test_save_default_config_includes_deployment(tmp_path):
1827     "compute_target": "",
1828     "max_concurrent_remote_jobs": 4,
1829     "remote_poll_max_wait_sec": 3900.0,
1830 +    "phase_placement": {"validation": None, "segment": None, "compile": None},
1831 +    "cloud_stitch_encoder": "h264_nvenc",
1832 }
1833     cfg = load_config(str(p))
1834     assert cfg.deployment.profile == ""
1835 diff --git a/tests/test_phase_placement.py b/tests/test_phase_placement.py
1836 new file mode 100644
1837 index 00000000..b9cc967
1838 --- /dev/null
1839 +++ b/tests/test_phase_placement.py
1840 @@ -0,0 +1,138 @@
1841 +"""Config-driven phase ? machine placement (#521).
1842 +
1843 +Proves the ``deployment.phase_placement`` knob + ``resolve_phase_placement`` resolver:
1844 the legacy
1845 +defaults reproduce today's placement exactly (validation + compile on the control-
1846 plane, segment on
1847 +the L4 iff compute_target==cloud_run), an explicit knob overrides, the cloud-serving
1848 preset places
1849 +compile on the L4, and the "nobody validates" footgun is rejected at config load.
1850 +"""
1851 +
1852 +import pytest
1853 +
1854 +from backend.config import (
1855 +    PHASE_COMPILE,
1856 +    PHASE_SEGMENT,
1857 +    PHASE_VALIDATION,
1858 +    PLACE_CLOUD_L4,
1859 +    PLACE_CONTROL_PLANE,
1860 +    AppConfig,
1861 +    load_config,
1862 +    resolve_phase_placement,
1863 +)

```

15. assistant (2026-07-08T06:05:31.233Z)

Now let me read the key post-#521 source files from the committed version (the working tree is a sibling's WIP without #521). Let me get the full `\_maybe\_dispatch\_remote`, `get\_compute\_backend`, and the compiler context.

16. user (2026-07-08T06:05:37.872Z)

done

17. user (2026-07-08T06:05:43.157Z)

```

199: def _cloud_available(cfg) -> bool:
218: def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
450: def _maybe_dispatch_remote(
751: def _execute_processing(
1190: def _stitch_reel(
1279: def _dispatch_compile_remote(
1423: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:

```

18. user (2026-07-08T06:05:46.527Z)

```

450     def _maybe_dispatch_remote(
451         cfg,
452         db,
453         req: "ProcessRequest",
454         video_id: str,
455         seg_name: str,
456         val_results,
457         play_wm: int,
458         cand_wm: int,
459         notify,
460     ) -> Optional[Tuple[Dict[str, Any], bool]]:
461         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
`deployment.compute_target
462         == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
463         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
464
465         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
466         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
467         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
468         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
469         the in-process path stays byte-identical)."""
470         from backend.compute import ComputeJobSpec, get_compute_backend
471
472         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
473             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
474             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
475             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
476             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
477             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
478             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
479             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
480             return (
481                 {
482                     "video_id": video_id,
483                     "status": "failed",
484                     "message": msg,
485                     "results": [r.model_dump() for r in val_results],
486                     "play_segments": [],
487                     "compute": {
488                         "path": "cloud_run" if ran else "not_run",
489                         "requested": req.compute,
490                         "target": "cloud_run",
491                         "dispatched": ran,
492                     },
493                 },

```

```

494         ran,
495     )
496
497     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
498     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
499     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
500     choice = (req.compute or "").strip().lower()
501     if choice and choice not in ("local", "cloud"):
502         logger.warning(
503             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
504             req.compute,
505         )
506         choice = ""
507     if choice == "local":
508         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
509         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
510         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
511         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
512         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
513         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
514         # below exactly as before (the weights-present check is the escape hatch).
515         if (
516             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
517             and _cloud_available(cfg)
518             and not _local_detector_weights_present(cfg, seg_name)
519         ):
520             return _failed(
521                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
522                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'." ,
523                 False,
524             )
525             return None # explicit "run on my machine" ? in-process regardless of the
server's target
526         if choice == "cloud":
527             if not _cloud_available(cfg):
528                 return _failed(
529                     "cloud-serving was requested but this server isn't configured for it "
530                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
531                     False,
532                 )
533             compute = get_compute_backend(cfg, target_override="cloud_run")
534         else:
535             # No explicit picker ? consult phase_placement.segment (#521). Its legacy
default is
536             # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
behaviour; setting
537             # phase_placement.segment relocates DETECT by config alone (e.g. force it back
on the
538             # control-plane, or onto the L4 on a default-local box wired for cloud).
539             if (
540                 resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
541                 and _cloud_available(cfg)

```

```

542         ):
543             compute = get_compute_backend(cfg, target_override="cloud_run")
544         else:
545             compute = None
546     if compute is None:
547         return None # default-OFF / placed on control_plane ? the in-process segmenter
below
548
549     storage_cfg = getattr(cfg, "storage", None)
550
551     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
552     try:
553         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
554     except ValueError as e:
555         return _failed(
556             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
557         )
558     if input_ref.backend == "local":
559         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
560         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
561         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
562         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
563         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
564         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
565         # default, while a bare/local-looking root can't stage no matter what
input_backend says
566         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
567         # gives us nothing to build the staged URI under).
568         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
569         try:

```

19. user (2026-07-08T06:05:55.049Z)

```

        )
        compute = get_compute_backend(cfg, target_override="cloud_run")
    else:
        compute = get_compute_backend(
            cfg
        ) # server default (default-OFF unless cloud_run)
    if compute is None:
        return None # default-OFF ? run the in-process segmenter below (unchanged)

    storage_cfg = getattr(cfg, "storage", None)

    # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
    try:
        input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
    except ValueError as e:
        return _failed(
            f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
        )
    if input_ref.backend == "local":
        # A browser upload lands on THIS box's local disk; the Cloud Run job (a different
machine)
        # can't read it. Stage it UP to the input bucket first, then dispatch from there (#461).
Only

```

```

        # possible when a CLOUD input_root is configured; without one we genuinely can't stage ?
fail.
        # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged URI below
is
        # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
        # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left at its
        # default, while a bare/local-looking root can't stage no matter what input_backend says
        # (resolve_input_uri only consults input_backend when input_root is EMPTY, and an empty
root
        # gives us nothing to build the staged URI under).
input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
try:
    root_backend = (
        storage.parse_uri(input_root).backend if input_root else "local"
    )
except ValueError: # unsupported scheme in input_root ? nowhere real to stage to
    root_backend = "local"
if root_backend == "local":

```

20. user (2026-07-08T06:06:06.153Z)

```

1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
      ``None``
5      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
      ``cpu_mock`` nothing
7      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8      ""
9
10     from __future__ import annotations
11
12     import dataclasses
13     import os
14     from typing import Any, Optional
15
16     from backend.compute.base import (
17         CompileJobSpec,
18         ComputeBackend,
19         ComputeJobSpec,
20         ComputeResult,
21         RemoteExecutionFailed,
22     )
23     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
24
25     __all__ = [
26         "ComputeBackend",
27         "ComputeJobSpec",
28         "CompileJobSpec",
29         "ComputeResult",
30         "RemoteExecutionFailed",
31         "get_compute_backend",
32     ]
33
34
35     def _storage_dict(config: Any) -> dict:
36         storage = getattr(config, "storage", None)
37         if storage is None:
38             return {}
39         if hasattr(storage, "model_dump"):

```

```

40         return storage.model_dump() # type: ignore[no-any-return]
41     return dict(storage) if isinstance(storage, dict) else {}
42
43
44     def get_compute_backend(
45         config: Any,
46         *,
47         run_client: Optional[Any] = None,
48         policy: ExecutionPolicy = DEFAULT_POLICY,
49         token: Optional[str] = None,
50         target_override: Optional[str] = None,
51     ) -> Optional[ComputeBackend]:
52         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
53         in-process (the default-OFF path).
54
55         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
56         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
57         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
58         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
59
60         ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
61         per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
62         server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
63         """
64         target = (
65             target_override
66             if target_override is not None
67             else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
68         )
69         if target != "cloud_run":
70             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
71
72         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
73         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
74
75         # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
76         # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
77         # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
old
78         # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
79         # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
80         if policy is DEFAULT_POLICY:
81             max_wait = getattr(
82                 getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
83             )
84             if max_wait:
85                 policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
86
87         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
88         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
89         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
90         client = run_client if run_client is not None else GoogleCloudRunJobClient()
91         return CloudRunCompute(
92             run_client=client,

```

```

93         job_name=job_name,
94         region=region,
95         project=project,
96         storage_config=_storage_dict(config),
97         policy=policy,
98         token=token,
99     )
100

```

21. assistant (2026-07-08T06:06:26.296Z)

Let me read `\_cloud\_available`, `\_will\_detect\_on\_cloud`, and how `out\_root` is consumed in the dispatch, plus the validation-gating in `\_execute\_processing`.

22. user (2026-07-08T06:06:29.059Z)

```

199     def _cloud_available(cfg) -> bool:
200         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
201 3).
202         True when the control plane is wired for Cloud Run: either the configured default
203 target is
204 already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
205 coordinates
206 (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
207 (``storage.output_root``)
208 are present so a forced per-request override can actually dispatch + collect a
209 result.
210
211 Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
212 would work."""
213     deployment = getattr(cfg, "deployment", None)
214     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
215         return True
216     has_job = bool(
217         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
218     )
219     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
220     return bool(has_job and out_root)
221
222     def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
223         """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
224         ``maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
225         ``execute_processing`` can
226         decide whether to DELEGATE validation to the worker
227         (``phase_placement.validation``). A per-request
228         picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
229         cloud iff
230         ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
231         pre-dispatch reject
232         (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
233         remotely."""
234         choice = (req_compute or "").strip().lower()
235         if choice == "local":
236             return False
237         if choice == "cloud":
238             return _cloud_available(cfg)
239         return (
240             resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
241             and _cloud_available(cfg)
242         )
243
244     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:

```

```
237         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
238         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
```

23. assistant (2026-07-08T06:07:21.753Z)

Let me read the full `\_execute\_processing` to verify the validation gating and `val\_results` initialization.

24. user (2026-07-08T06:07:25.069Z)

```
751     def _execute_processing(
752         config=None, db=None, req=None, progress=None
753     ) -> Dict[str, Any]:
754         """The full hash ? validate ? segment ? report pipeline for one video.
755
756         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
757         module-level ``global_config``/``db_instance`` (backward-compatible with existing
758         tests that pass only ``req``). Route handlers always pass explicit params; older
759         test code that calls ``_execute_processing(req)`` still works.
760
761         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
762         Raises on unexpected internal errors ? the caller records those as a job failure
763         """
764         import backend.main as _main # call-time seam: main's globals/model stay patchable
765
766         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
767         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
768         if config is not None and not isinstance(config, AppConfig):
769             # Old signature: _execute_processing(req, progress=None)
770             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
771                 req = db
772                 db = None
773             elif req is None and isinstance(config, _main.ProcessRequest):
774                 req = config
775                 config = None
776         # Resolve config/db from module globals when not passed explicitly
777         if config is None:
778             config = _main.global_config
779         if db is None:
780             db = _main.db_instance
781
782         notify = progress or (lambda stage: None)
783
784         notify("hashing")
785         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
786         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
787         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
788         # consumers (hash / validators / segmenter) use the resolved local path.
789         local_video = ""
790         try:
791             local_video = storage.acquire_local_input(
792                 req.video_path, getattr(config, "storage", None)
793             )
794             video_id = _main.compute_video_id(local_video)
795         except Exception:
796             storage.cleanup_acquired_local_input(
797                 req.video_path, local_video, getattr(config, "storage", None)
798             )
799             raise
```

```

800     try:
801         existing_video = db.get_video(video_id)
802     except Exception:
803         storage.cleanup_acquired_local_input(
804             req.video_path, local_video, getattr(config, "storage", None)
805         )
806         raise
807     was_reingest = existing_video is not None
808
809     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
810     default_seg = getattr(
811         getattr(config, "indexing", None), "default_segmenter", "motion"
812     )
813     seg_name = req.segmenter or default_seg
814     segmenter_actual = None
815     segmentation_ran = False
816     val_results: List[Any] = []
817     val_context: Dict[str, Any] = {}
818     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
819     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
820     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
821     compute_actual: Optional[str] = None
822     response: Optional[Dict[str, Any]] = None
823     try:
824         if (
825             existing_video
826             and existing_video.get("status") == "processed"
827             and not req.force
828         ):
829             cached_segments = db.query_play_segments(video_id, {})
830             if cached_segments:
831                 logger.info(
832                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
833                     video_id,
834                 )
835             response = _cached_process_payload(
836                 video_id,
837                 existing_video.get("validation_results", []),
838                 cached_segments,
839                 getattr(req, "compute", None),
840             )
841             return response
842
843     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
844     #     UNLESS this exact content already has a cached verdict under the current
validator config.
845     notify("validating")
846     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
847     # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
848     # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
849     # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
850     # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
851     # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
852     # here). The fingerprint folds in the validation.* config + sport + validator

```

```

set + a schema
853         # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
854         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
855         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
856         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
857         #
858         # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
859         # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
860         # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
861         # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
862         # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
863         # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
864         # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
865         validate_on_control_plane = (
866             resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
867             or not _will_detect_on_cloud(config, getattr(req, "compute", None))
868         )
869         if not validate_on_control_plane:
870             logger.info(
871                 "[API] validation delegated to the L4 worker "
872                 "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
873                 "validator suite; the worker is the gate for %s.",
874                 req.video_path,
875             )
876         val_fingerprint = (
877             registry.config_fingerprint(config) if validate_on_control_plane else ""
878         )
879         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
880         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
881         # failing the job. The cache is a pure speedup ? never a new failure mode.
882         reused_validation = False
883         if validate_on_control_plane and not req.force:
884             try:
885                 cached = db.get_validation_cache(video_id)
886                 if cached is not None and cached.get("fingerprint") == val_fingerprint:
887                     # Replay the stored ValidationResult list; val_context stays {} (no
fresh
888                     # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
889                     # deliberate cost of skipping the decode). Segmentation reads
neither of these.
890                     val_results = [
891                         ValidationResult(**r) for r in cached.get("results", [])
892                     ]
893                     reused_validation = True
894                     logger.info(
895                         "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
896                         "validator config unchanged ? skipping the validator suite. "
897                         "force=true re-validates.",
898                         cached.get("passed"),

```

```

899         video_id,
900     )
901     except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
902         reused_validation = False
903         logger.warning(
904             "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
905             video_id,
906             exc_info=True,
907         )
908     if validate_on_control_plane and not reused_validation:
909         logger.info(f"Running validations for {req.video_path}...")
910         val_results, val_context = registry.run_all_with_context(local_video,
config)
911     failures = [r for r in val_results if not r.passed]
912     if validate_on_control_plane and not reused_validation:
913         # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
914         # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
915         db.set_validation_cache(
916             video_id,
917             val_fingerprint,
918             not failures,
919             [r.model_dump() for r in val_results],
920         )
921
922     if failures:
923         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
924         # status; it no longer destroys the video's prior good segments.
925         db.add_video(
926             video_id,
927             req.video_path,
928             "failed",
929             [r.model_dump() for r in val_results],
930         )
931         response = {
932             "video_id": video_id,
933             "status": "failed",
934             "message": _validation_failure_message(failures),
935             "results": [r.model_dump() for r in val_results],
936         }
937         return response
938
939     # 2. Run segmenter & indexer
940     logger.info("[API] Video passed checks. Segmenting and indexing...")
941     db.add_video(
942         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
943     )
944
945     segmenter_cls = segmenter_registry.get(seg_name)
946     if not segmenter_cls:
947         # Submit-time validation already 400s unknown names; this is the defensive
948         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
949         available = list(segmenter_registry.list_available().keys())
950         response = {
951             "video_id": video_id,
952             "status": "failed",
953             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
954             "results": [r.model_dump() for r in val_results],
955             "play_segments": [],
956         }

```

```

957         return response
958
959         logger.info(f"[API] Using segmenter: {seg_name}")
960         notify(f"segmenting ({seg_name})")
961         segmenter_actual = seg_name
962         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
963         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
964         play_wm, cand_wm = db.segment_watermarks(video_id)
965         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
966         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
967         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
968         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
969         # they're dropped on the cloud path in this slice ? a documented follow-up.)
970         remote = _maybe_dispatch_remote(

```

25. assistant (2026-07-08T06:07:38.739Z)

Let me verify the compiler imports (the new code uses `Dict`, `Tuple`, `subprocess`, `ffmpeg\_timeout\_sec`, `ffmpeg\_bin`) and check `compile\_highlights` for the concat path.

26. user (2026-07-08T06:07:41.822Z)

```

1     import logging
2     import os
3     import subprocess
4     import tempfile
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
9         ffmpeg_bin,
10        ffmpeg_timeout_sec,
11        ffprobe_bin,
12        ffprobe_timeout_sec,
13        timeout_label,
14    )
15
16    logger = logging.getLogger(__name__)
17
18    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21    # backend/pipeline/stitcher/, so parents[2] == backend/.
22    _BUNDLED_FONT = (

```

27. user (2026-07-08T06:07:46.432Z)

```

515         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
516             logger.info(
517                 f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
518             )
519             # Build the (optional) watermark filtergraph once; it is applied during
every
520             # per-clip re-encode so the mark is baked into each clip identically.

```

```

521         watermark_filter = self._watermark_filter(tmp_dir)
522         if watermark_filter:
523             logger.info("[watermark] applying branding overlay to each clip.")
524         # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
525         # and the final concat stays -c copy (#521).
526         video_codec_args, chosen_encoder = self._select_encoder()
527         if chosen_encoder != "libx264":
528             logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
529         for idx, (raw_start, raw_end) in enumerate(segments):
530             clip_path, skip_record = self._trim_one_segment(
531                 idx,
532                 raw_start,
533                 raw_end,
534                 segments,
535                 video_duration,
536                 tmp_dir,
537                 raw_video_path,
538                 watermark_filter,
539                 video_codec_args,
540             )
541             if clip_path is not None:
542                 temp_clips.append(clip_path)
543             if skip_record is not None:
544                 skipped_segments.append(skip_record)
545
546         # Report summary of skipped segments
547         if skipped_segments:
548             logger.info(
549                 f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
550             )
551             for seg_idx, seg_start, seg_end, reason in skipped_segments:
552                 logger.info(
553                     f" - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
554                 )
555             logger.info("")
556
557         if not temp_clips:
558             raise RuntimeError(
559                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
560                 f"All {len(segments)} segments were skipped. "
561                 f"Check the errors above for details. "
562                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
563             )
564
565         logger.info(
566             f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
567         )
568
569         # Create a list file for FFmpeg concat demuxer
570         list_file_path = os.path.join(tmp_dir, "clips.txt")
571         # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
572         # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
573         with open(list_file_path, "w", encoding="utf-8") as f:
574             for clip in temp_clips:
575                 # The concat demuxer resolves relative entries against the list
file's
576                 # directory. Use absolute paths so a temp-dir clip does not become
577                 # output/tmp.../output/tmp.../clip_0.mp4 on Windows.

```

```

578         safe_path = Path(clip).resolve().as_posix().replace("'", "'\\'")
579         f.write(f"file '{safe_path}'\n")
580
581     logger.info("Stitching clips together...")
582     # Stitch them together without re-encoding (since they were encoded with
identical settings)
583     concat_cmd = [
584         ffmpeg_bin(),
585         "-y",
586         "-f",
587         "concat",
588         "-safe",
589         "0",
590         "-i",
591         list_file_path,
592         "-c",
593         "copy",
594         output_path,
595     ]
596
597     try:
598         timeout = ffmpeg_timeout_sec()
599         subprocess.run(
600             concat_cmd, capture_output=True, check=True, timeout=timeout
601         )
602
603         # Final verification
604         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:

```

28. user (2026-07-08T06:08:03.664Z)

```

379         else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
380     )
381     base_cmd = [
382         ffmpeg_bin(),
383         "-y",
384         "-ss",
385         str(round(padded_start, 3)),
386         "-to",
387         str(round(padded_end, 3)),
388         "-i",
389         raw_video_path,
390         *codec_args,
391         "-c:a",
392         "aac",
393         "-b:a",
394         "128k",
395     ]
396     vf_args = ["-vf", watermark_filter] if watermark_filter else []
397
398     # Try with the watermark first; if that encode fails (e.g. a bad font/logo
399     # path), retry once WITHOUT it so a branding misconfig can never nuke the
400     # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
401     attempts = [vf_args, []] if vf_args else [[]]
402     produced = False
403     last_exc: Optional[subprocess.CallProcessError] = None
404     cmd = base_cmd + [clip_path]
405     timeout = ffmpeg_timeout_sec()
406     for attempt_vf in attempts:
407         cmd = base_cmd + attempt_vf + [clip_path]
408         try:
409             subprocess.run(
410                 cmd, capture_output=True, check=True, timeout=timeout
411             )
412             except subprocess.TimeoutExpired:

```

```

413         reason = f"ffmpeg timeout after {timeout_label(timeout)}"
414         logger.error(f"{segment_label}: FFmpeg trim timed out.")
415         print(f"  Command: {' '.join(cmd)}")
416         print(f"  {reason}")
417         return None, (idx, start, end, reason)
418     except subprocess.CalledProcessError as e:
419         last_exc = e
420         if attempt_vf and vf_args:
421             _err = (
422                 e.stderr.decode(errors="replace").strip()[-300:]
423                 if getattr(e, "stderr", None)
424                 else ""
425             )
426             logger.warning(
427                 f"{segment_label}: watermark encode failed; retrying without it."
428                 f"ffmpeg: {_err or '(no stderr captured)'}"
429             )
430             continue
431         break
432     if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
433         produced = True
434         if not attempt_vf and vf_args:
435             logger.warning(
436                 f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
437             )
438         break
439     last_exc = None # ffmpeg succeeded but produced nothing
440     break
441
442     if produced:
443         return clip_path, None
444     elif last_exc is not None:
445         stderr_msg = last_exc.stderr.decode(errors="replace").strip()
446         logger.error(f"{segment_label}: FFmpeg trim failed.")
447         print(f"  Command: {' '.join(cmd)}")
448         print(
449             f"  FFmpeg stderr: {stderr_msg[-500:]}")
450         ) # Last 500 chars to avoid flooding
451         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
452     else:
453         logger.error(
454             f"{segment_label}: FFmpeg exited successfully but output clip is missing
or empty. Skipping."
455         )
456         return None, (idx, start, end, "empty output clip")
457
458     def compile_highlights(
459         self,
460         raw_video_path: str,
461         segments: List[Tuple[float, float]],
462         output_filename: str,
463     ) -> str:
464         """
465         Extracts selected segments from a raw video and stitches them together
466         into a seamless highlights file, applying end_padding to prevent abrupt endings.
467
468         .. warning::
469             This method uses a two-stage codec contract: per-clip re-encodes must use
470             IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
471             concat can stream-copy (`-c copy`) without re-encoding. Changing the
472             clip encode settings without updating the concat assumptions will produce
473             broken output.
474         """

```

```

475         if not segments:
476             raise ValueError("No highlight segments provided to compile.")
477
478         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
479         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
480         original_count = len(segments)
481         segments = self._merge_overlapping(segments)
482         if len(segments) < original_count:
483             logger.info(
484                 "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
%d.",
485                 original_count - len(segments),
486                 original_count,
487                 len(segments),
488             )
489         if not segments:
490             raise ValueError("No valid (start < end) highlight segments after merge.")
491
492         if not os.path.exists(raw_video_path):
493             raise FileNotFoundError(
494                 f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
495             )
496
497         # Defense-in-depth: never let the output name escape output_dir (the API also
498         # validates this at the request boundary).

```

29. assistant (2026-07-08T06:08:25.105Z)

Let me read the full `\_stitch\_reel` and `\_dispatch\_compile\_remote` and `\_execute\_compile` to compare against the pre-521 in-process stitch.

30. user (2026-07-08T06:08:27.781Z)

```

1190 def _stitch_reel(
1191     cfg,
1192     stitching,
1193     video_id: str,
1194     source_ref: str,
1195     time_pairs: List[Tuple[float, float]],
1196     out_name: str,
1197     locale: Optional[str],
1198     notify,
1199 ) -> Dict[str, Any]:
1200     """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201     (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202     needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203     with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204     trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205     returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206     ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1207     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209     output_dir = (
1210         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211     )
1212     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);

```

```

1213     # unknown/absent locale keeps the configured default unchanged.
1214     localized_watermark = localize_watermark(stitching.watermark, locale)
1215     compiler = _main.VideoCompiler(
1216         output_dir,
1217         begin_padding=stitching.begin_padding,
1218         end_padding=stitching.end_padding,
1219         watermark=localized_watermark,
1220         encoder=getattr(stitching, "encoder", "libx264"),
1221     )
1222
1223     notify("stitching")
1224     local_src = ""
1225     try:
1226         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1227         # path). Resolve to
1228         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1229         # scratch for a URI (the
1230         # same seam the process path uses); without it, compiling a bucket-ingested
1231         # video can't open gs://.
1232         local_src = storage.acquire_local_input(
1233             source_ref, getattr(cfg, "storage", None)
1234         )
1235         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
1236         # (for remote
1237         # dispatch) doesn't run multiple compiles at once on the box (see
1238         # _LOCAL_COMPUTE_LANE).
1239         with _LOCAL_COMPUTE_LANE:
1240             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1241         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
1242             # FAILURE, not a crash
1243             logger.error(
1244                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1245             )
1246         return {
1247             "status": "failed",
1248             "message": f"Highlight stitching failed: {e}",
1249             "video_id": video_id,
1250         }
1251     finally:
1252         if local_src:
1253             storage.cleanup_acquired_local_input(
1254                 source_ref, local_src, getattr(cfg, "storage", None)
1255             )
1256
1257     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
1258     # URL ? Cloud Run
1259     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
1260     # copy dies when the
1261     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
1262     # (the endpoint
1263     # still falls back to the local file on the same instance). On the L4 worker the
1264     # mirror is how the
1265     # reel reaches the bucket at all (the worker's local file is discarded when the Job
1266     # exits).
1267     reel_uri = reel_object_uri(cfg, video_id, out_name)
1268     if reel_uri:
1269         try:
1270             notify("uploading reel")
1271             # Thread the active storage settings so the configured backend (S3
1272             # endpoint/region, GCS
1273             # project/client) is used, not a fresh default.
1274             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1275         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
1276             # already succeeded
1277             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

```

```

1265
1266     # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1267     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1268     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1269     notify("finished")
1270     return {
1271         "status": "success",
1272         "filepath": out_path,
1273         "download_url": download_url,
1274         "video_id": video_id,
1275         "reel_uri": reel_uri or "",
1276     }
1277
1278
1279     def _dispatch_compile_remote(

```

31. assistant (2026-07-08T06:08:31.737Z)

Let me compare with the pre-521 `\_execute\_compile` to confirm the in-process stitch is byte-identical.

32. user (2026-07-08T06:08:34.613Z)

```

def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
    reel,
    and return the {status, filepath, download_url} result the SPA polls for. Returns a
    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a stitch
    error)
    so the SPA always gets a clean verdict instead of a worker-crash error."""
    import backend.main as _main # call-time seam: VideoCompiler stays patchable on
    backend.main

    notify = progress or (lambda stage: None)
    params = json.loads(job.get("params") or "{}")
    video_id = job.get("video_id") or ""
    segment_ids = params.get("segment_ids") or []
    locale = params.get("locale")

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, params.get("output_filename", "")
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    output_dir = (
        getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
    )
    # Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
    job);
    # unknown/absent locale keeps the configured default unchanged.
    localized_watermark = localize_watermark(stitching.watermark, locale)
    compiler = _main.VideoCompiler(
        output_dir,
        begin_padding=stitching.begin_padding,
        end_padding=stitching.end_padding,
        watermark=localized_watermark,
    )
    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

    notify("stitching")

```

```

    local_src = ""
    try:
        # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local path).
        Resolve to
        # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-scratch for a
        URI (the
        # same seam the process path uses); without it, compiling a bucket-ingested video can't
        open gs://.
        local_src = storage.acquire_local_input(
            video["filepath"], getattr(cfg, "storage", None)
        )
        # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count (for
        remote
        # dispatch) doesn't run multiple compiles at once on the box (see _LOCAL_COMPUTE_LANE).
        with _LOCAL_COMPUTE_LANE:
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job FAILURE, not
        a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }
    finally:
        if local_src:
            storage.cleanup_acquired_local_input(
                video["filepath"], local_src, getattr(cfg, "storage", None)
            )

    # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed URL ?
    Cloud Run
    # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp copy dies
    when the
    # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch (the
    endpoint
    # still falls back to the local file on the same instance).
    reel_uri = reel_object_uri(cfg, video_id, out_name)
    if reel_uri:
        try:
            notify("uploading reel")
            # Thread the active storage settings so the configured backend (S3 endpoint/region,
            GCS
            # project/client) is used, not a fresh default.
            storage.put(out_path, reel_uri, config=storage_settings(cfg))
        except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch already
        succeeded
            logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

    # `filepath` is server-local (not browser-reachable); also return the download URL so the
    SPA needn't
    # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
    download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
    notify("finished")
    return {
        "status": "success",
        "filepath": out_path,
        "download_url": download_url,
        "video_id": video_id,
    }
}

```

Let me check all `return None` and `return \_failed` paths within `\_maybe\_dispatch\_remote` (after the compute is obtained) to confirm it never silently falls through to the in-process segmenter when validation was delegated.

34. user (2026-07-08T06:09:09.189Z)

```
2:         return None # default-OFF / placed on control_plane ? the in-process segmenter below
10:         return _failed(
31:             return _failed(
58:             return _failed(
72:         return _failed(
142:         return _failed(f"cloud dispatch error: {e}", True)
144:         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
165:         return _failed(
193:     return (
206: def _execute_processing(
```

35. user (2026-07-08T06:09:17.500Z)

```
970         remote = _maybe_dispatch_remote(
971             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
972         )
973         if remote is not None:
974             response, segmentation_ran = (
975                 remote # `ran` = did the cloud job actually execute (report)
976             )
977             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
978         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
979         compute_actual = "in_process"
980         segmenter = segmenter_cls(db, config)
981         try:
982             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
983             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
984             with _LOCAL_COMPUTE_LANE:
985                 result = segmenter.process_video(
986                     video_id, local_video, req.player_pool, req.max_frames
987                 )
988         except Exception:
989             # A raising segmenter must not leave half-written rows: restore the pre-run
990             # state (same destroy-after-confirm contract as the Failure branch), then
let
991             # the job worker record the failure.
992             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
993             raise
994         segmentation_ran = True
995
996         if isinstance(result, Failure):
997             logger.error(f"[API] Segmenter failed: {result.message}")
998             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
999             response = {
1000                 "video_id": video_id,
1001                 "status": "failed",
1002                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
1003                 "results": [r.model_dump() for r in val_results],
1004                 "play_segments": [],
1005             }
1006             return response
1007
1008         segments = result.value if hasattr(result, "value") else result
1009         notify("finalizing")
```

36. assistant (2026-07-08T06:11:21.203Z)

I've completed a thorough regression analysis of the #521 diff against its parent (`699d85b`), reading the post-change source directly from the merged commit (`8866ffe`) since the working tree holds a sibling session's unrelated WIP that I left untouched.

Let me do a final verification of the two subtlest invariants by confirming the `\_cloud\_available` short-circuit makes the new segment gate a no-op for the legacy default.

37. user (2026-07-08T06:11:24.814Z)

```
=== OLD default seg branch (699d85b) ===
510:         ) # server default (default-OFF unless cloud_run)
=== _cloud_available short-circuit on compute_target==cloud_run ===
    deployment = getattr(cfg, "deployment", None)
    if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
        return True
    has_job = bool(
        os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
    )
    out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
    return bool(has_job and out_root)
```

38. assistant (2026-07-08T06:11:46.640Z)

My regression review of the #521 diff is complete. I read the full diff (`8866ffe` vs parent `699d85b`) and the post-change source directly from the merged commit, leaving the sibling session's dirty working tree untouched.

Verdict for the regression dimension: no defects. All four invariants hold byte-identically on the default-OFF path:

- Unset `phase\_placement` + `compute\_target=''` ? `resolve\_phase\_placement(cfg, compile)` returns `control\_plane`, so `\_execute\_compile` skips the remote branch and stitches in-process via `\_stitch\_reel`, which reproduces the old `\_execute\_compile` body exactly (only additive `reel\_uri` key; `VideoCompiler` gets `encoder=libx264` by default). `resolve\_phase\_placement(cfg, validation)` returns `control\_plane`, so `validate\_on\_control\_plane=True` and every new `if validate\_on\_control\_plane and ?` guard collapses to the original condition ? validation, caching, and persistence run unchanged.
- `\_maybe\_dispatch\_remote` default branch dispatches iff `resolve(segment)==cloud\_run\_l4` and `\_cloud\_available(cfg)`. For unset `phase\_placement.segment`, `resolve==cloud\_run\_l4` ? `compute\_target=='cloud\_run'`, and `\_cloud\_available` short-circuits to `True` whenever `compute\_target=='cloud\_run'` (orchestration.py:209-210), so the added `\_cloud\_available` gate is a no-op there. The branch dispatches exactly when `compute\_target=='cloud\_run'`, identical to pre-521. `get\_compute\_backend(cfg, target\_override='cloud\_run')` returns the same `CloudRunCompute` as the old `get\_compute\_backend(cfg)` in that case.
- Validation can never be skipped with nobody validating. `validate\_on\_control\_plane` is False only when validation is on L4 *and* `\_will\_detect\_on\_cloud` is True. `\_will\_detect\_on\_cloud==True` is exactly the condition under which `\_maybe\_dispatch\_remote` obtains a non-None `compute`; I verified every subsequent exit in that function is a `\_failed(...)` or the success tuple ? there is no `return None` fall-through to the in-process segmenter after the compute is acquired (orchestration.py:547 is the last such None). And with validation on L4, `worker\_skips\_validation = (resolve(validation) != cloud\_run\_l4) = False`, so the worker is the gate. The CP-skip and worker-skip conditions are mutually exclusive. The config-load guard (`\_validation\_placement\_needs\_cloud\_segment`) plus the runtime `\_will\_detect\_on\_cloud` (compute='local' forces CP validation) both hold.
- `VideoCompiler.\_trim\_one\_segment` with `video\_codec\_args=None` falls back to `["-c:v","libx264","-preset","superfast","-crf","22"]` spliced at the same argv position as the old inline literal, and `\_select\_encoder()` for the default libx264 returns that identical list with no probe subprocess. The concat stays `-c copy`; `\_select\_encoder` is called once before the clip loop so the encoder choice is all-or-nothing.

Now returning the structured result.

39. user (2026-07-08T06:11:46.709Z)

Structured output provided successfully